Foundations of RDF Databases

Claudio Gutierrez

Department of Computer Science
Universidad de Chile

European Semantic Web Conference - ESWC 2008

Joint Work With

- Renzo Angles
- Marcelo Arenas
- Carlos Hurtado
- Sergio Muñoz
- Jorge Pérez

Inspired by…

To the memory of

Alberto Mendelzon,

database theoretician
and Web enthusiast

Agenda

1. RDF and Databases
2. RDF and Database models
3. RDF Query Language
   - Requirements and Domains
   - Manifold Views
4. SPARQL
   - Syntax and Semantics
   - Complexity
   - Expressive Power

Agenda

1. RDF and Databases
2. RDF and Database models
3. RDF Query Language
   - Requirements and Domains
   - Manifold Views
4. SPARQL
   - Syntax and Semantics
   - Complexity
   - Expressive Power

Disclaimers

More like a “Computer Science” than a “Web Science” talk



Apologies to Web Scientists

A particular view on the subject



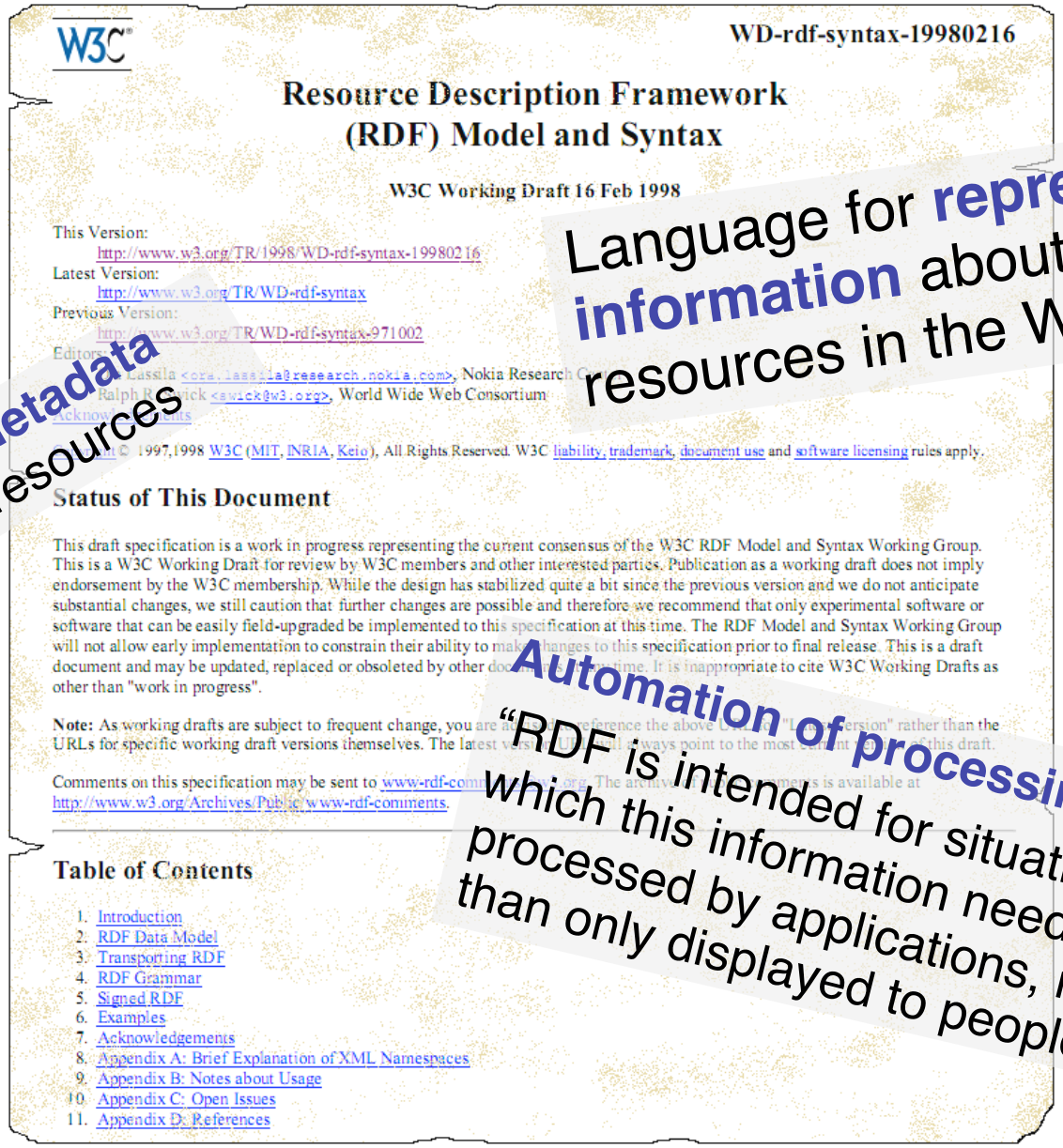
Not a survey!

The base of the Semantic Web is RDF

"The **Semantic Web** is the representation of **data** on the World Wide Web. It is a collaborative effort led by W3C with participation from a large number of researchers and industrial partners. **It is based on the Resource Description Framework (RDF)**"

http://www.w3.org/2001/sw/

RDF Recommendation (1999)



W3C

WD-rdf-syntax-19980216

Resource Description Framework (RDF) Model and Syntax

W3C Working Draft 16 Feb 1998

This Version:
http://www.w3.org/TR/1998/WD-rdf-syntax-19980216
Latest Version:
http://www.w3.org/TR/WD-rdf-syntax
Previous Version:
http://www.w3.org/TR/WD-rdf-syntax-971002
Editors:
Ora Lassila <ora.lassila@research.nokia.com>, Nokia Research
Ralph R. Swick <swick@w3.org>, World Wide Web Consortium
Acknowledgments

Copyright © 1997,1998 W3C (MIT, INRIA, Keio), All Rights Reserved. W3C liability, trademark, document use and software licensing rules apply.

Status of This Document

This draft specification is a work in progress representing the current consensus of the W3C RDF Model and Syntax Working Group. This is a W3C Working Draft for review by W3C members and other interested parties. Publication as a working draft does not imply endorsement by the W3C membership. While the design has stabilized quite a bit since the previous version and we do not anticipate substantial changes, we still caution that further changes are possible and therefore we recommend that only experimental software or software that can be easily field-upgraded be implemented to this specification at this time. The RDF Model and Syntax Working Group will not allow early implementation to constrain their ability to make changes to this specification prior to final release. This is a draft document and may be updated, replaced or obsoleted by other documents at any time. It is inappropriate to cite W3C Working Drafts as other than "work in progress".

Note: As working drafts are subject to frequent change, you are ... reference the above ... version" rather than the URLs for specific working draft versions themselves. The latest ... always point to the most current ... of this draft.

Comments on this specification may be sent to www-rdf-com... The archive ... comments is available at http://www.w3.org/Archives/Public/www-rdf-comments.

Table of Contents

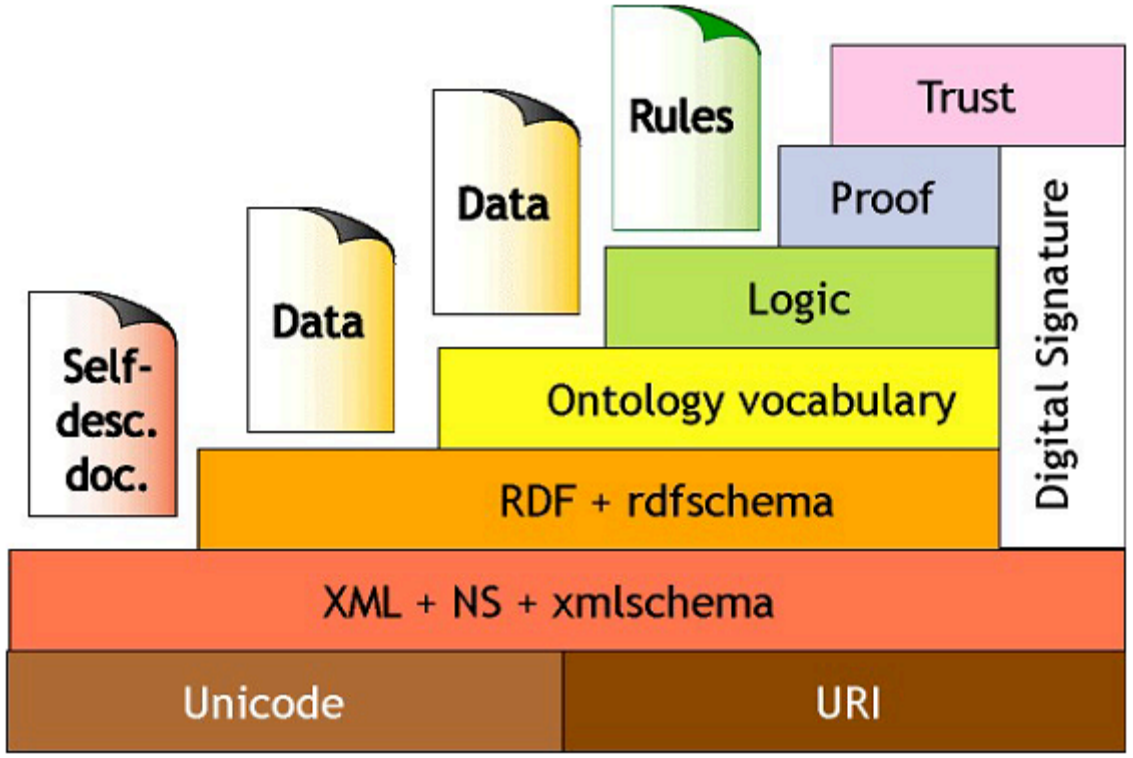
1. Introduction
2. RDF Data Model
3. Transporting RDF
4. RDF Grammar
5. Signed RDF
6. Examples
7. Acknowledgements
8. Appendix A: Brief Explanation of XML Namespaces
9. Appendix B: Notes about Usage
10. Appendix C: Open Issues
11. Appendix D: References

Particularly **metadata** about Web resources

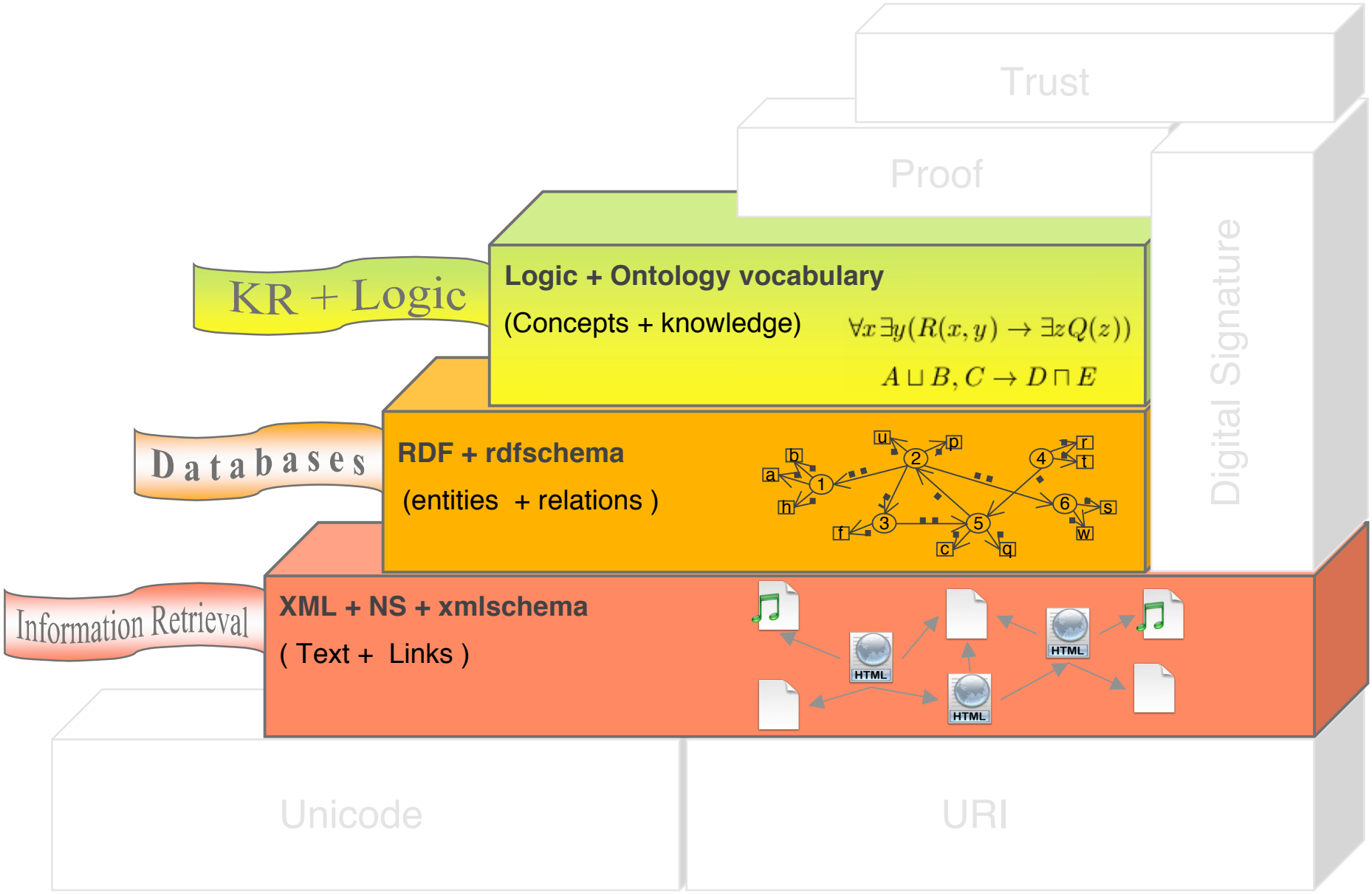
Language for **representing information** about resources in the Web

Automation of processing:
"RDF is intended for situations in which this information needs to be processed by applications, rather than only displayed to people"

Layers of the Semantic Web

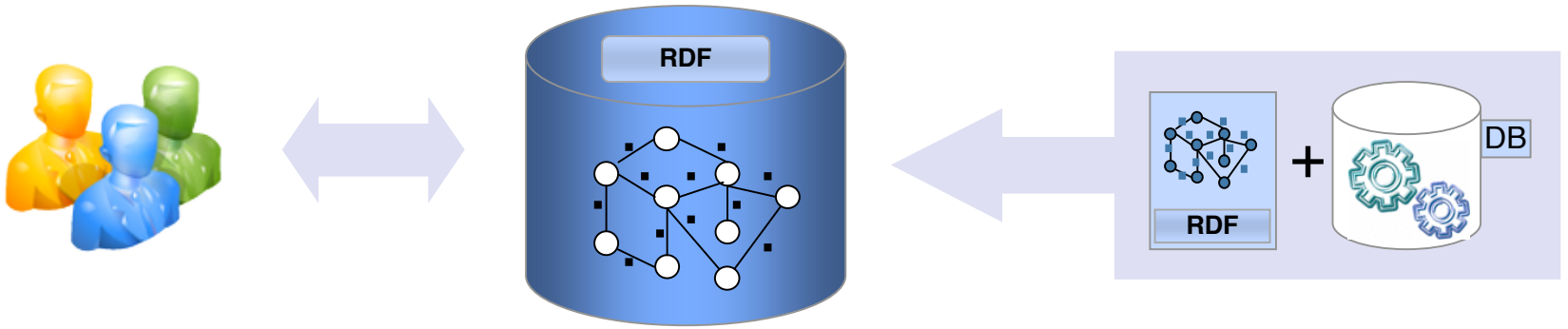


A Data Processing perspective



The Database Approach

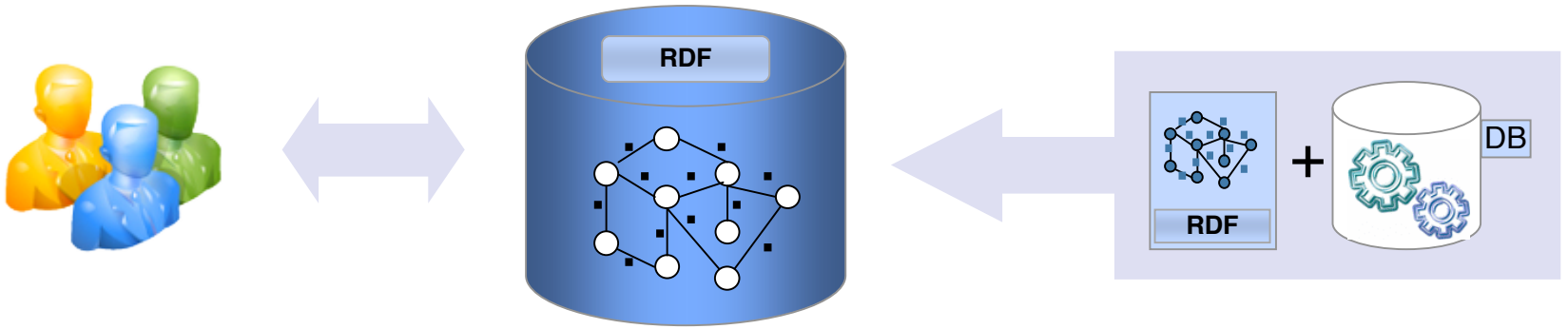
- Manage huge volumes of data with logical precision
- Separate modeling from implementation levels



The Database Approach

- Manage huge volumes of data with logical precision
- Separate modeling from implementation levels

As opposed to AI: DB primary concern is scalability. Then expressive power

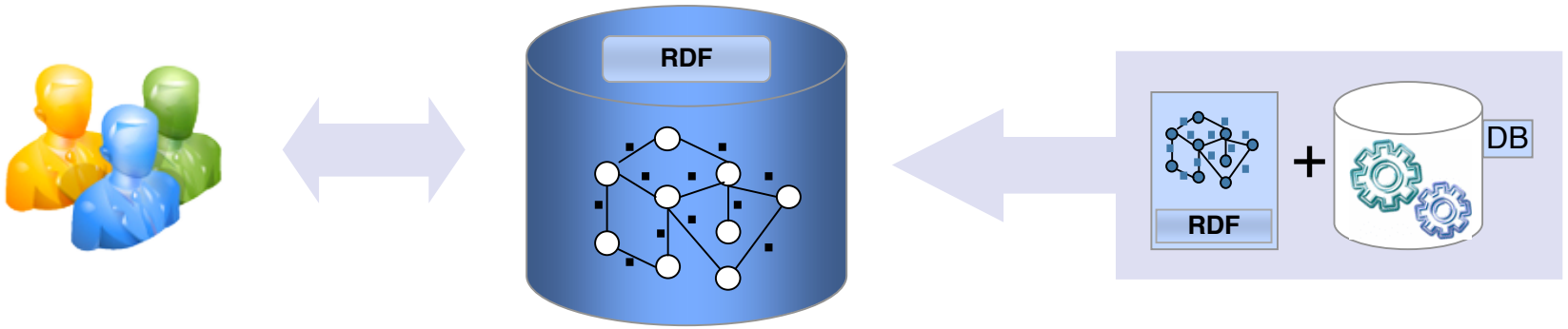


The Database Approach

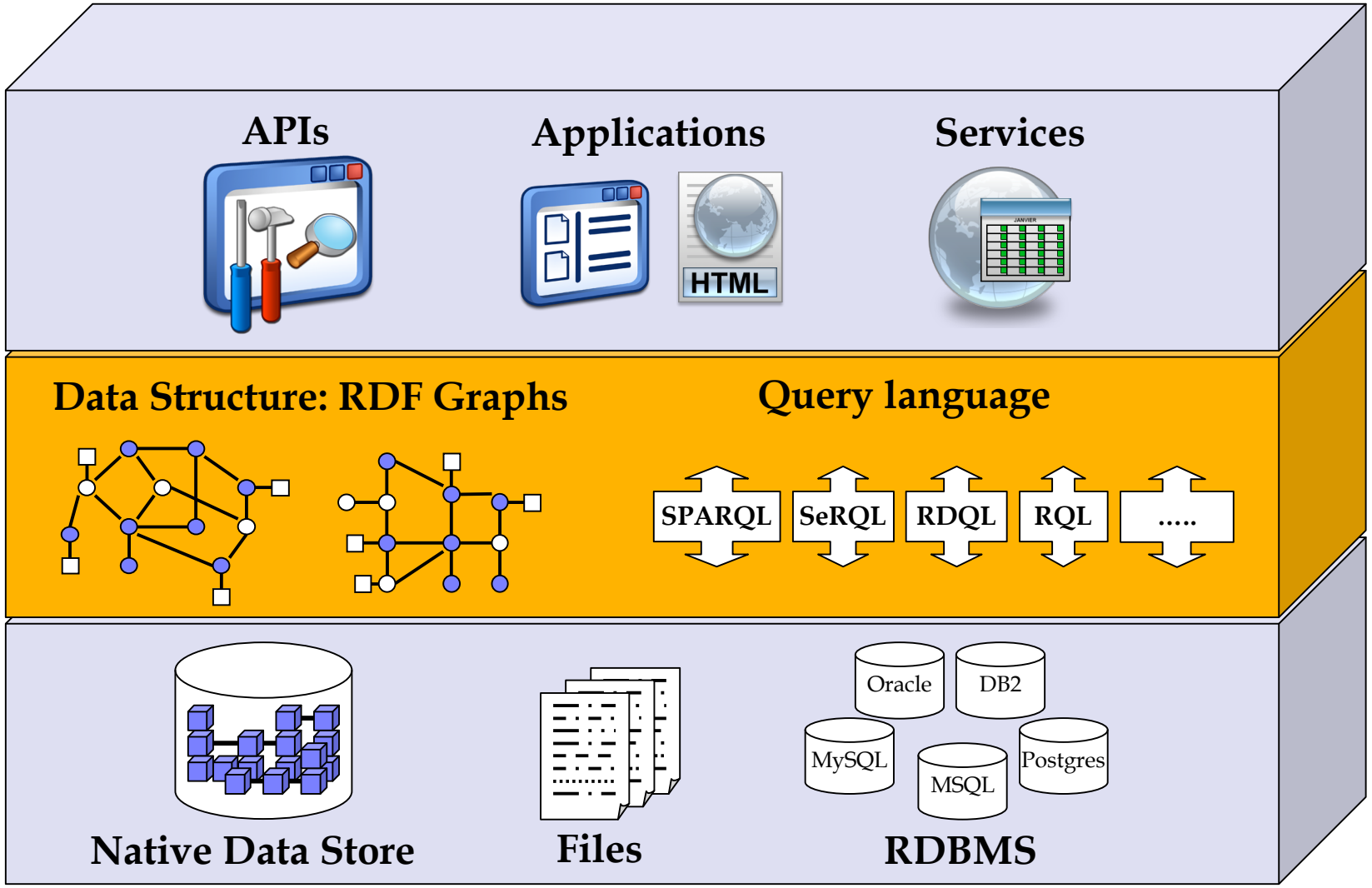
- Manage huge volumes of data with logical precision
- Separate modeling from implementation levels

As opposed to AI: DB primary concern is scalability. Then expressive power

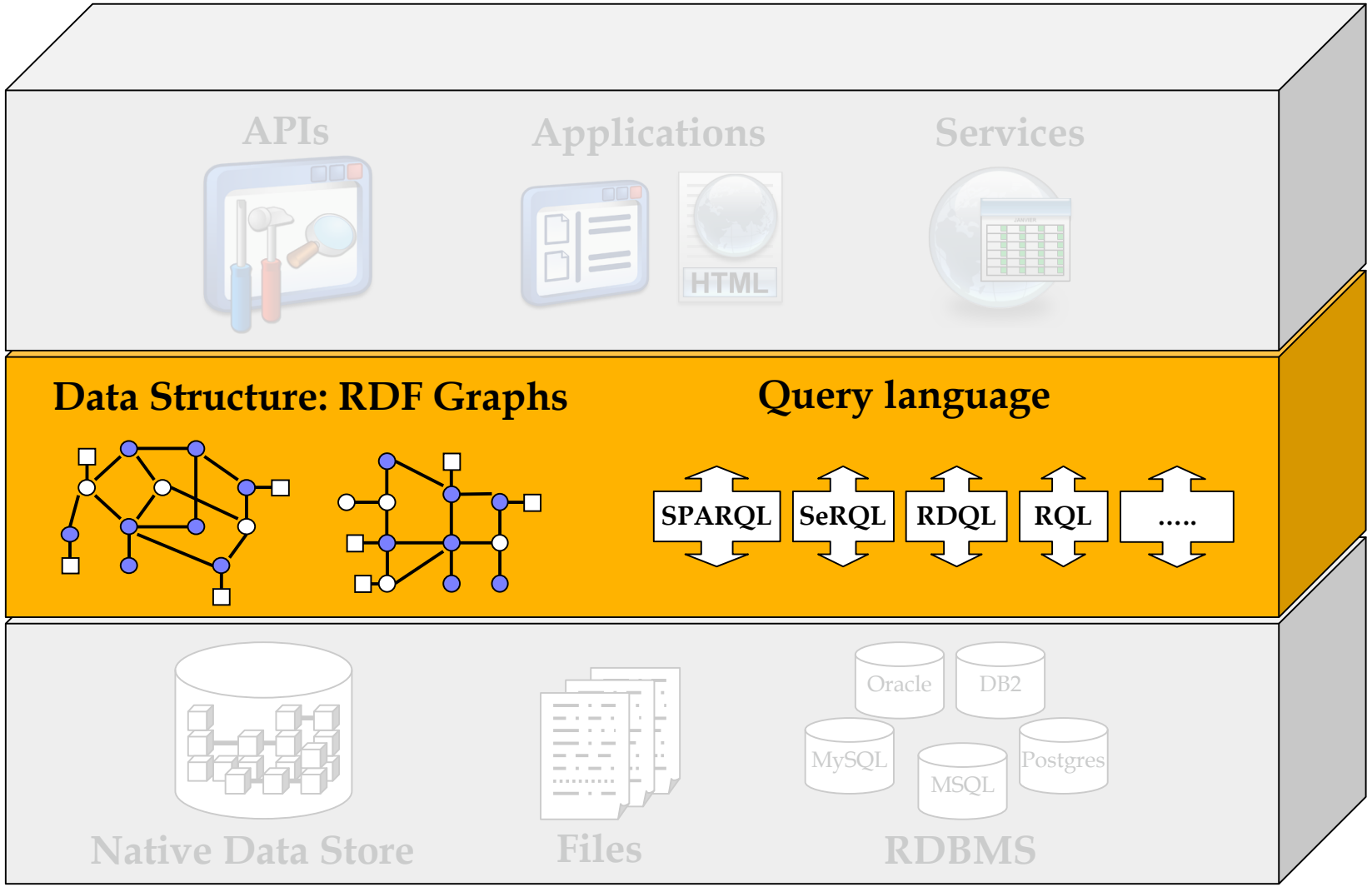
As opposed to IR: DB primary concern is precision. Then scalability (recall).



RDF Database Technology



This Talk: Database Modeling Level



This Talk: Database Modeling Level

Hence leaving out:

- Visualization, APIs, Services, etc.
- Indexing, storing, transactions, etc.

This Talk: Database Modeling Level

Hence leaving out:

- Visualization, APIs, Services, etc.
- Indexing, storing, transactions, etc.

But also leaving out:
Updating / Constraints / Temporality / Optimization / Aggregation / Flexibility / etc. / etc.

Agenda

1. RDF and Databases
2. RDF and Database models
3. RDF Query Language
   - Requirements and Domains
   - Manifold Views
4. SPARQL
   - Syntax and Semantics
   - Complexity
   - Expressive Power

Database Models: Codd's definition

Query Language

Integrity constraints

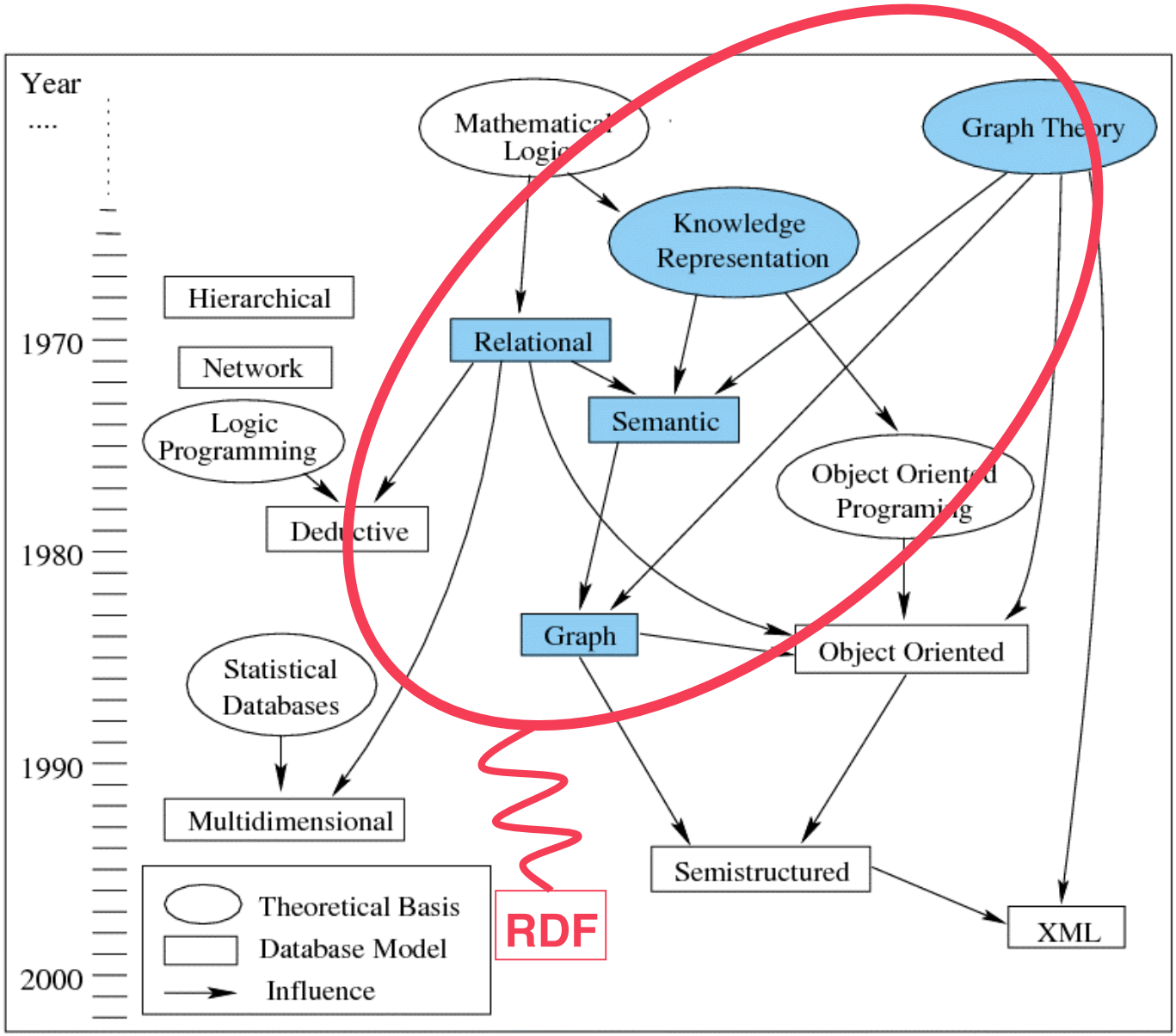
Data structures

Database Models: Codd's definition

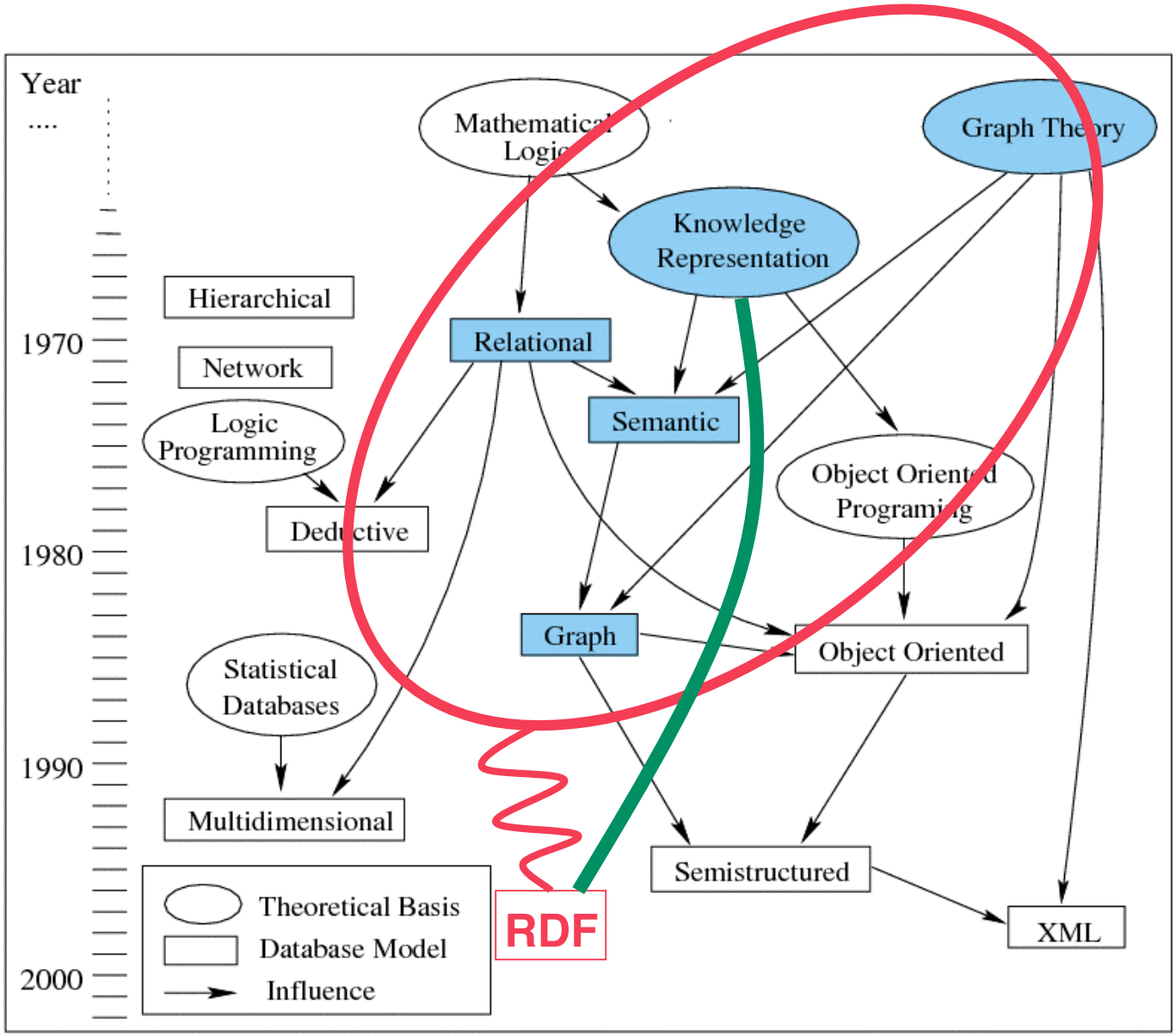
Query Language

Data structures

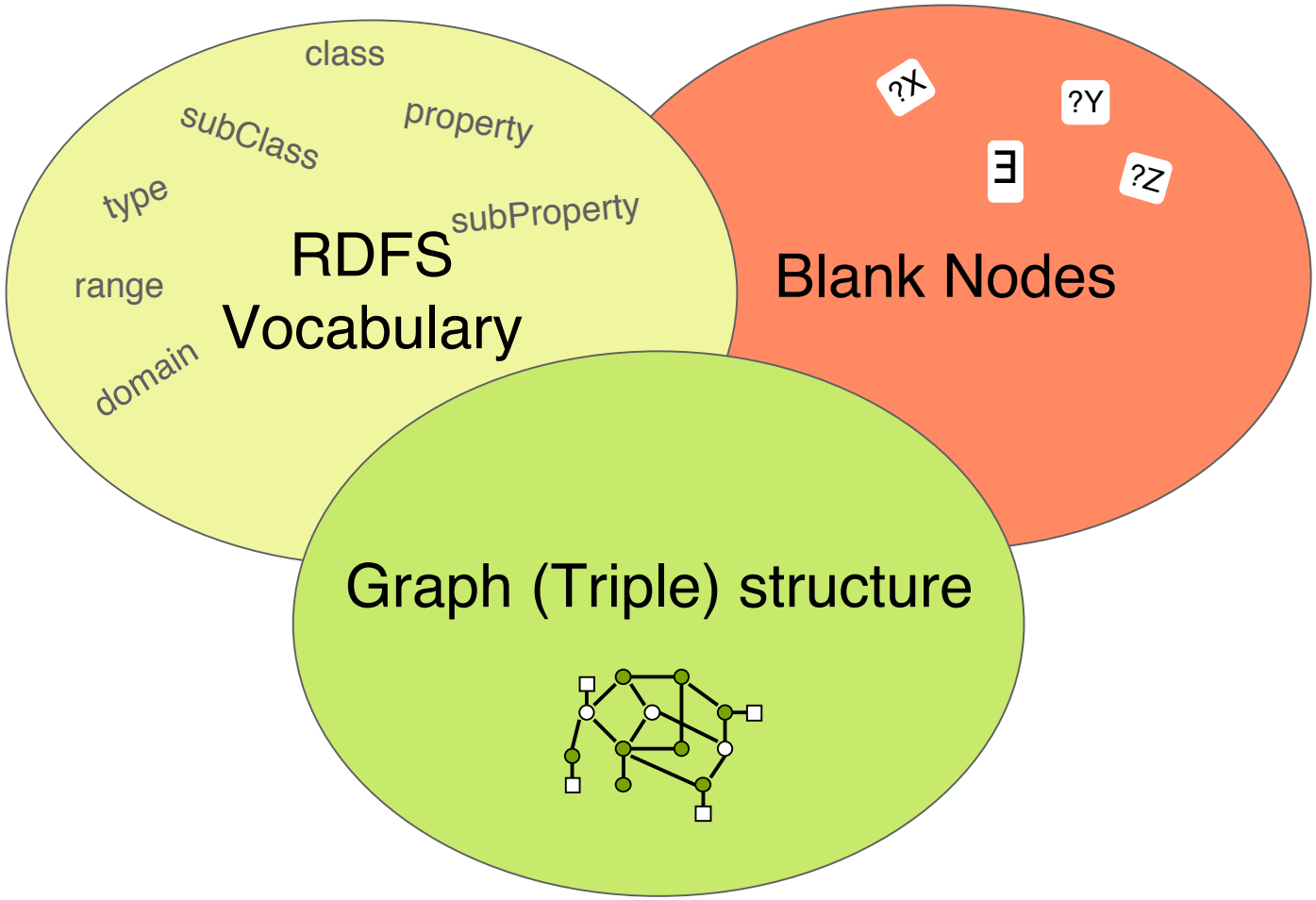
Evolution of Database Models



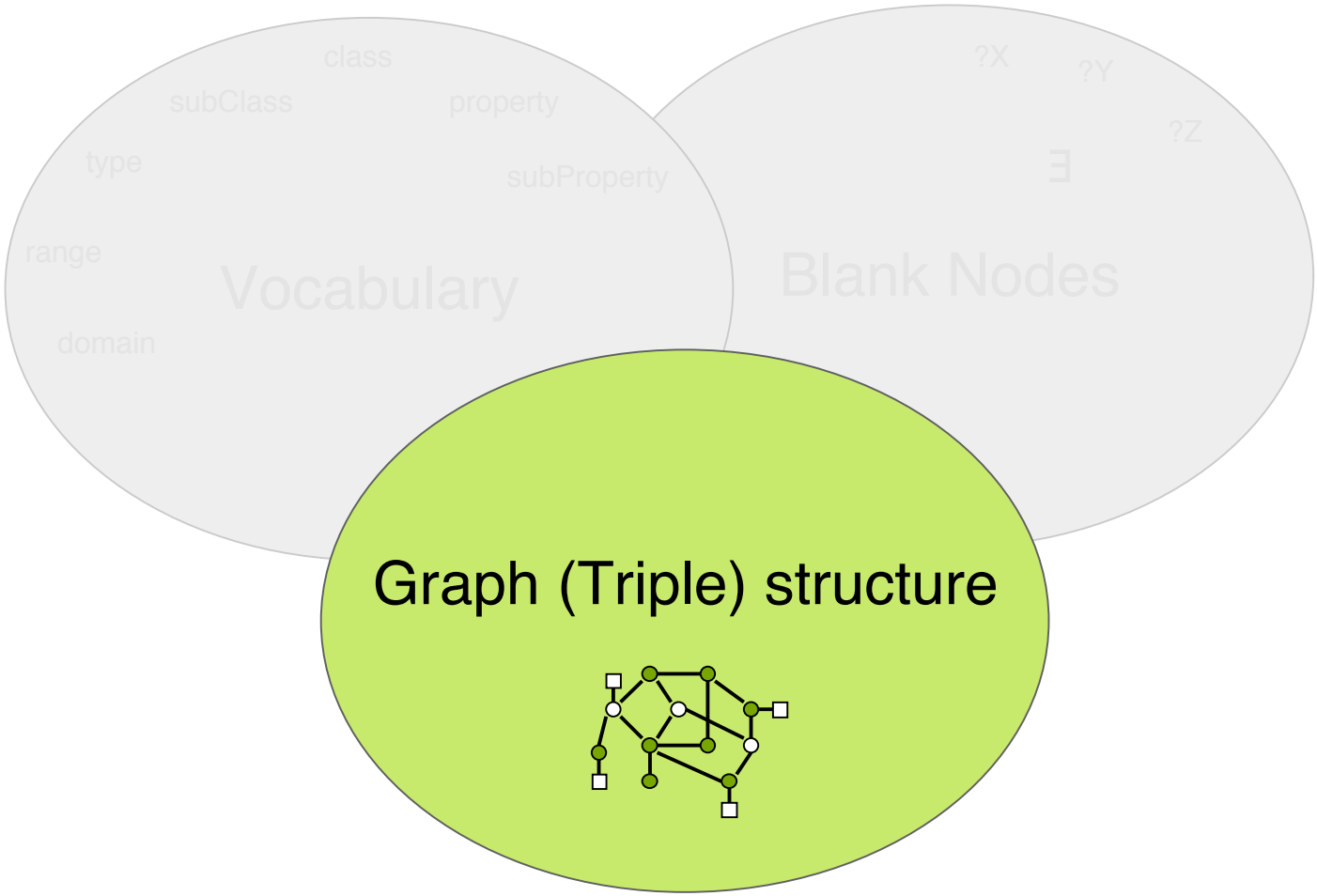
Evolution of Database Models



RDF Data Structure: three main blocks

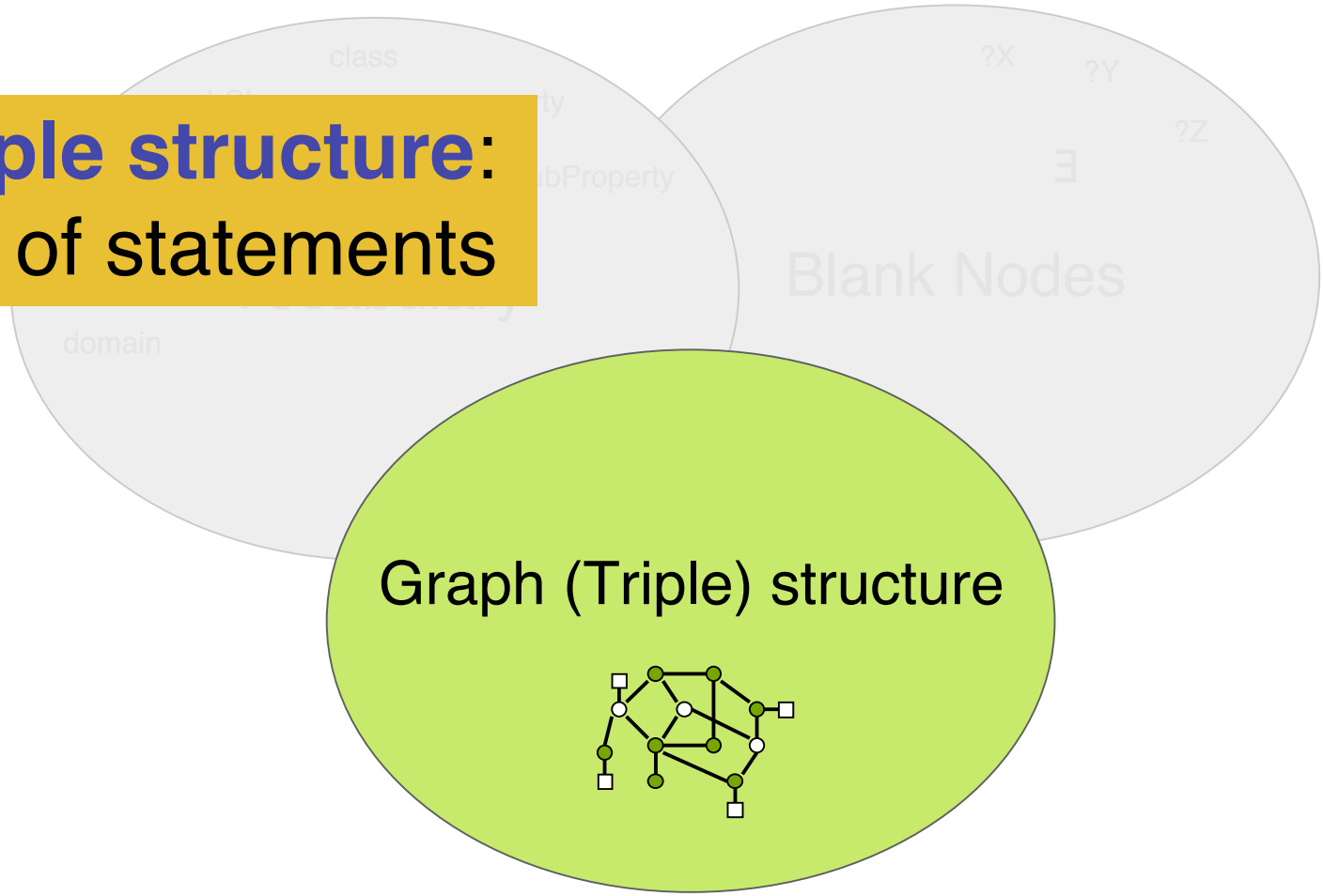


RDF Data Structure: the core



RDF Data Structure: the core

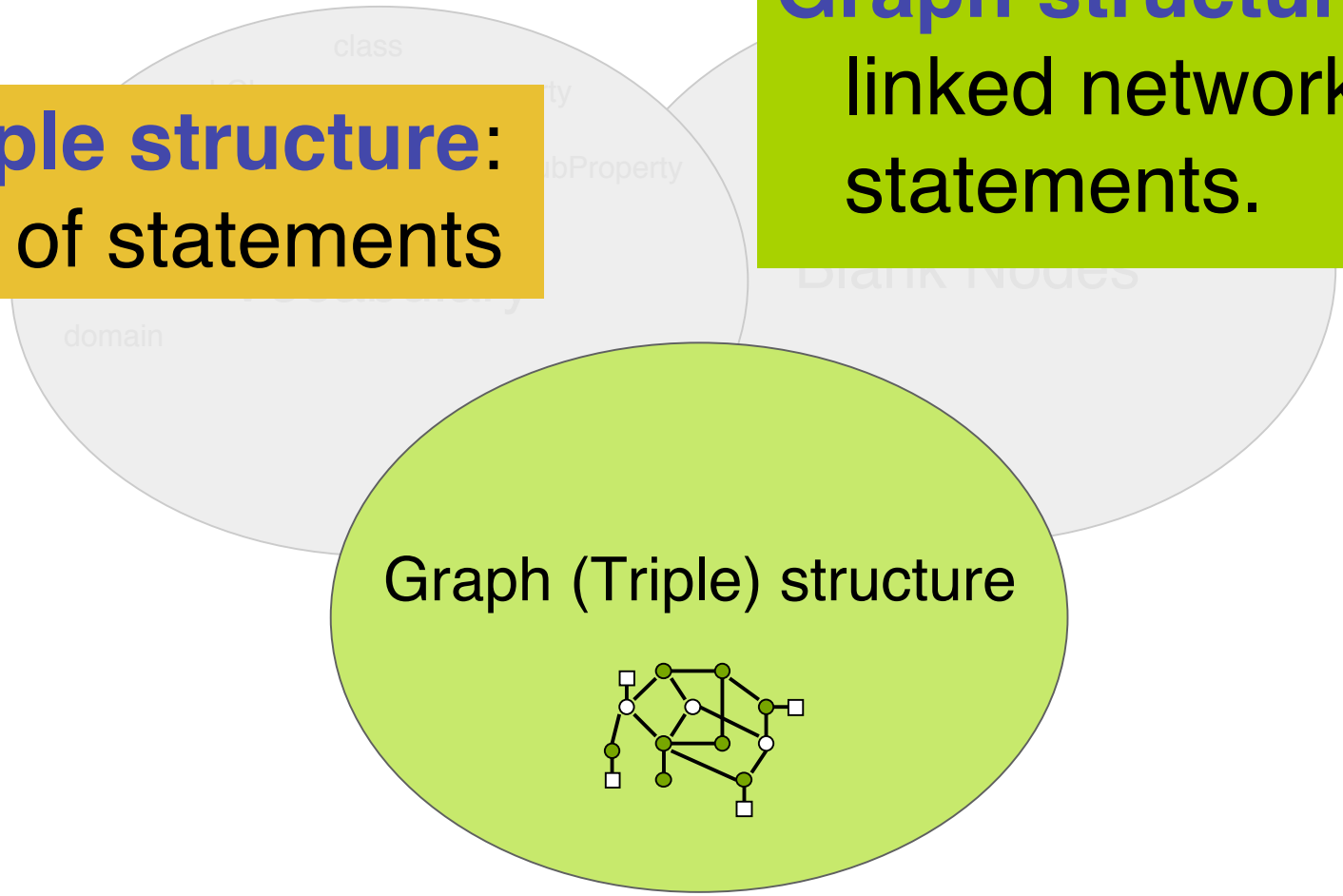
Triple structure:
set of statements



RDF Data Structure: the core

Triple structure:
set of statements

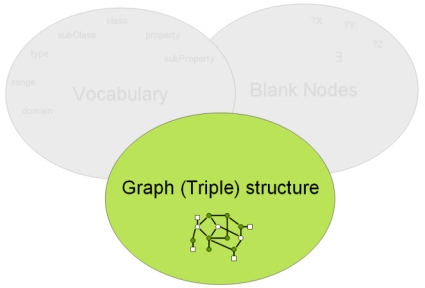
Graph structure:
linked network of statements.



RDF Data Structure: Relational Tables (Triple) view

- Triples as tuples
- Set of triples as Tables

Subject	Predicate	Object



RDF Data Structure: Relational Tables (Triple) view

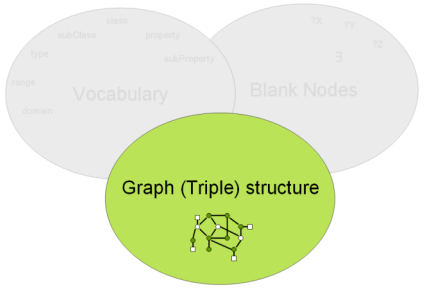
- Triples as tuples
- Tables of triples

Subject	Predicate	Object

Advantages:

+ Well studied and well understood

+ Reuse relational technologies



RDF Data Structure: Relational Tables (Triple) view

- Triples as tuples
- Tables of triples

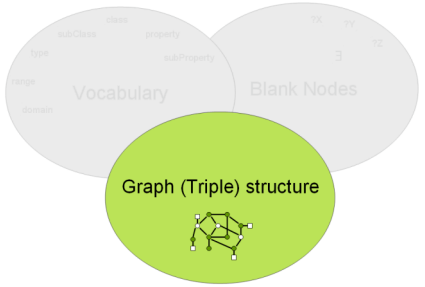
Subject	Predicate	Object

Advantage

+ Well studie
  well und
+ Reuse relat
  technolog

Problems (Questions):

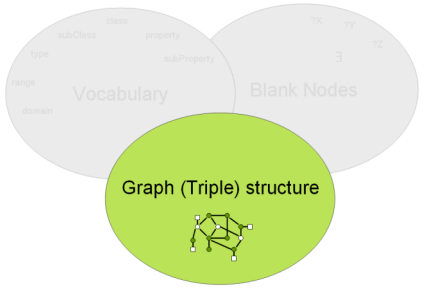
- Why yet another syntax for the relational model?
- Was this the intended objective of RDF?
- Expressive power limitations



RDF Data Structure: Graph Database Model view

Graph Database Models:

- Data and/or schema are represented by graphs
- Query language able to capture main graph operations and properties
- Studied by DB community, but still not well understood



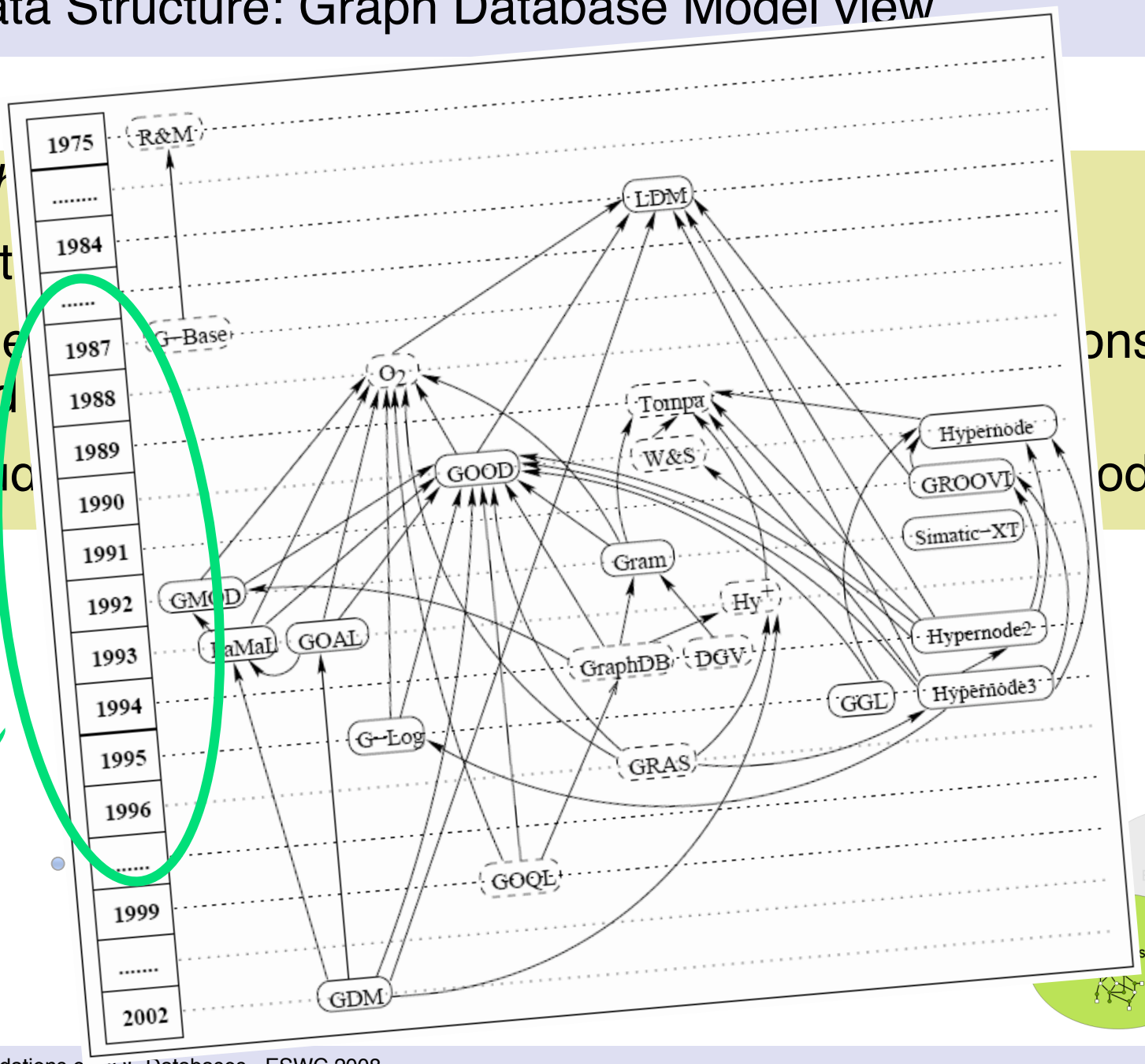
RDF Data Structure: Graph Database Model view

Graph

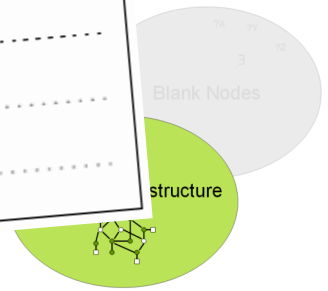
- Dat
- Que and
- Stuc

ons

od



Golden age

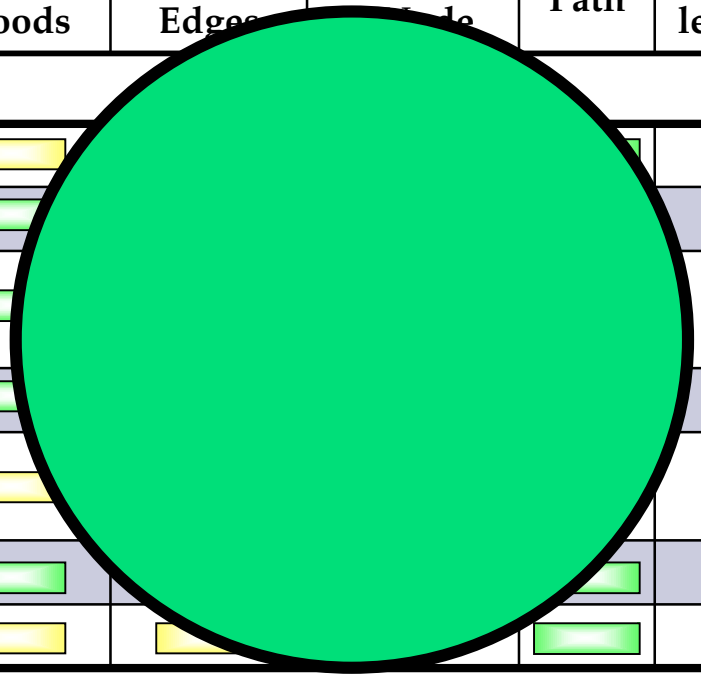


RDF Data Structure: Graph query languages

PROPERTY		Neighborhoods	Adjacent Edges	Degree of a Node	Path	Fixed-length path	Distance	Diameter
Graph Query Language	G							
	G+							
	Graph Log							
	Gram							
	Graph DB							
	Lorel							
	F-G							

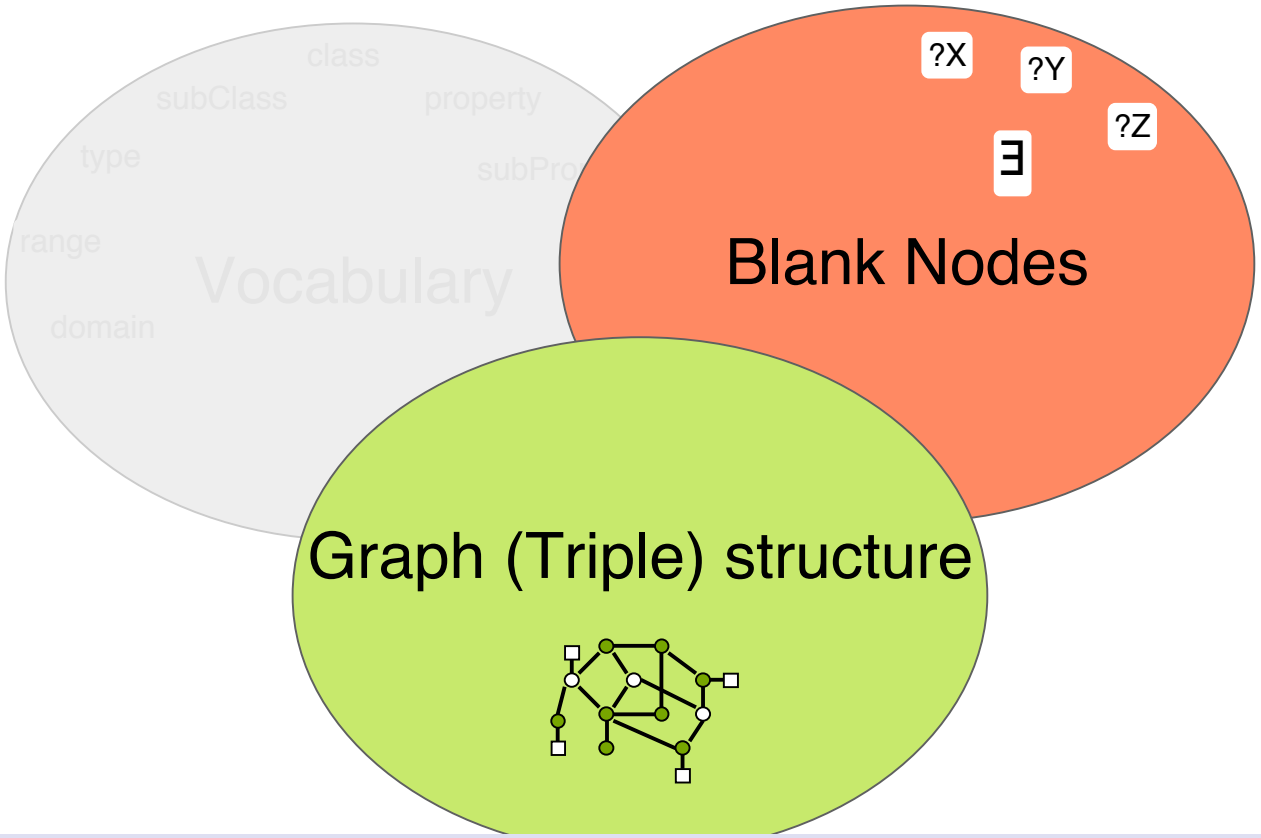
RDF Data Structure: Graph query languages

PROPERTY		Neighbo rhoods	Adjacent Edge	Degree of	Path	Fixed-length path	Distance	Diameter
Graph Query Language	G							
	G+							
	Graph Log							
	Gram							
	Graph DB							
	Lorel							
	F-G							



Green light for graph features!

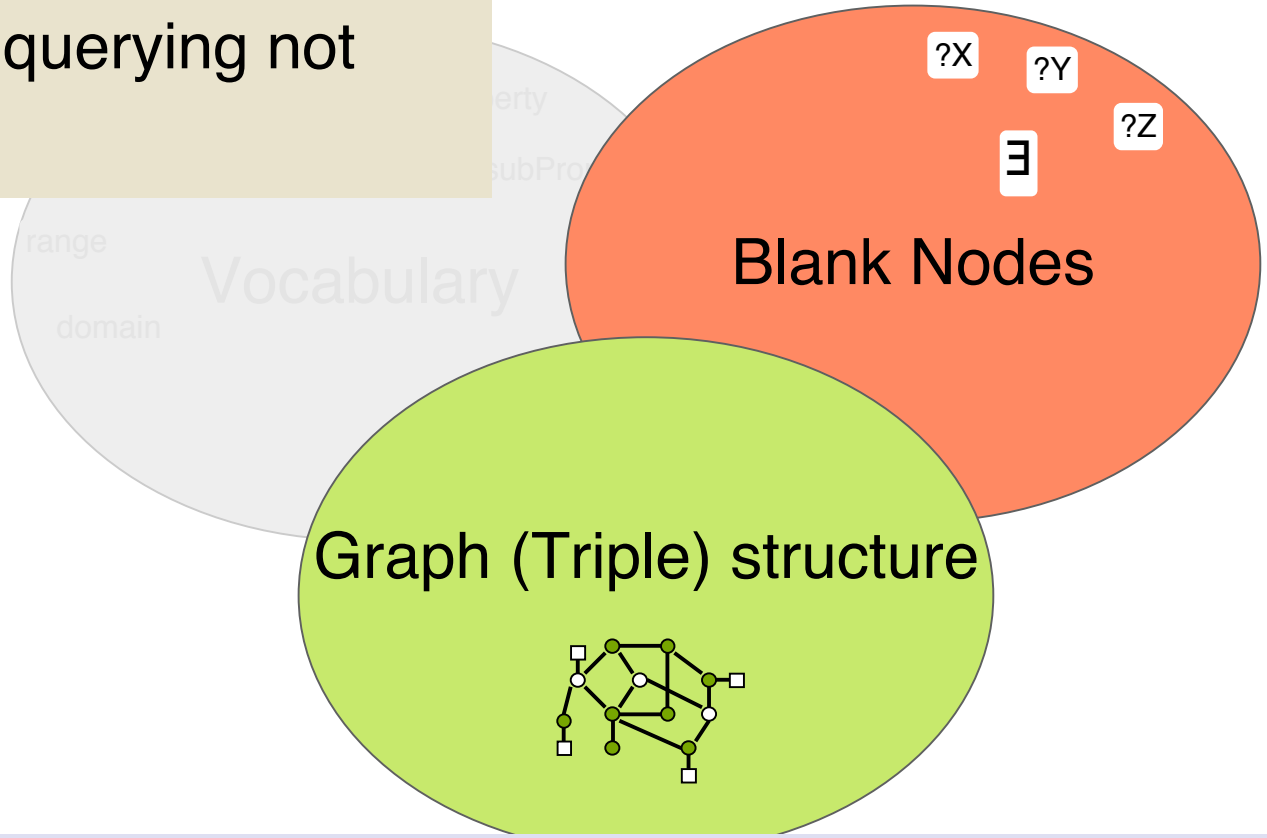
RDF Data Structure: Triple structure + Blank nodes



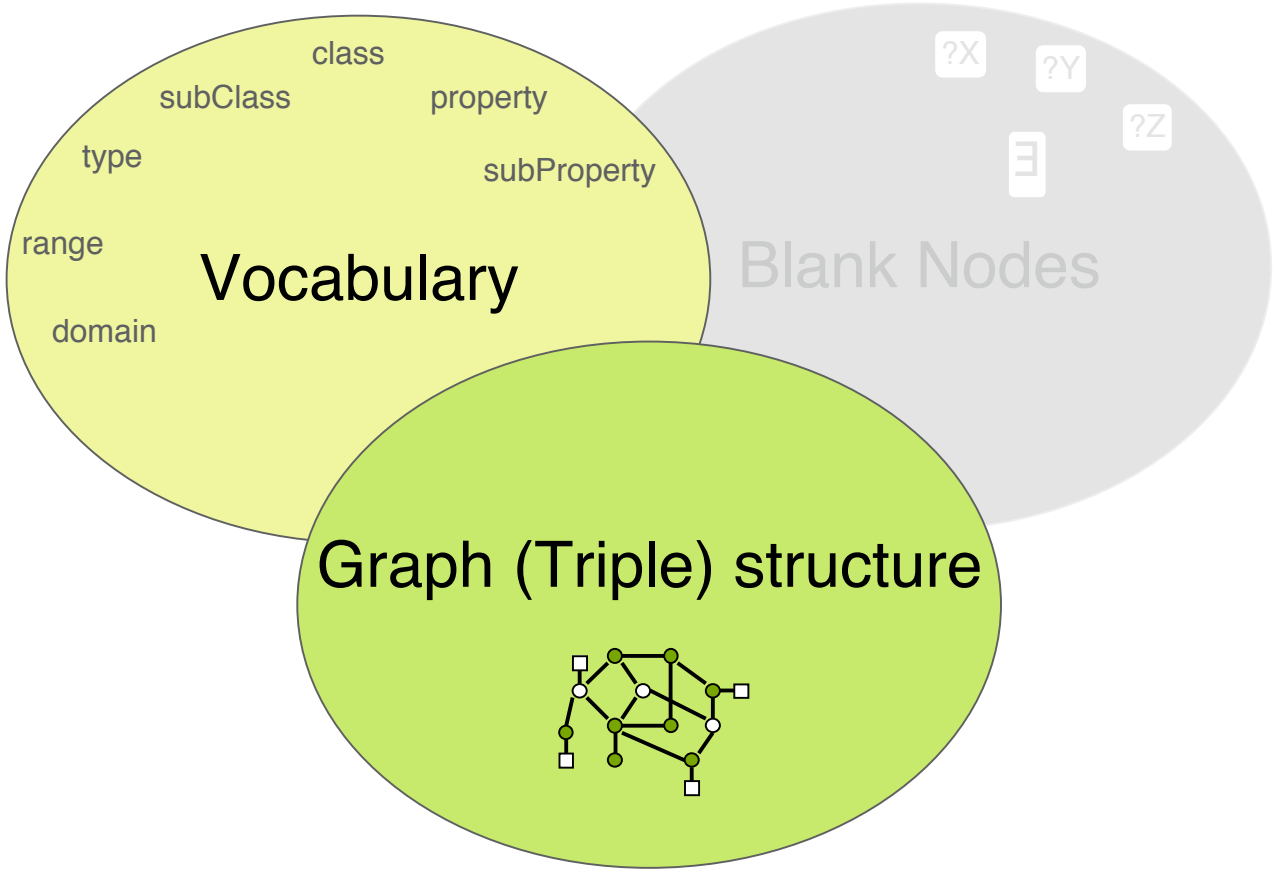
RDF Data Structure: Triple structure + Blank nodes

Complexity / Semantics issues:

- Deciding entailment becomes NP-complete.
- Deciding core is DP-complete
- Semantics of querying not simple

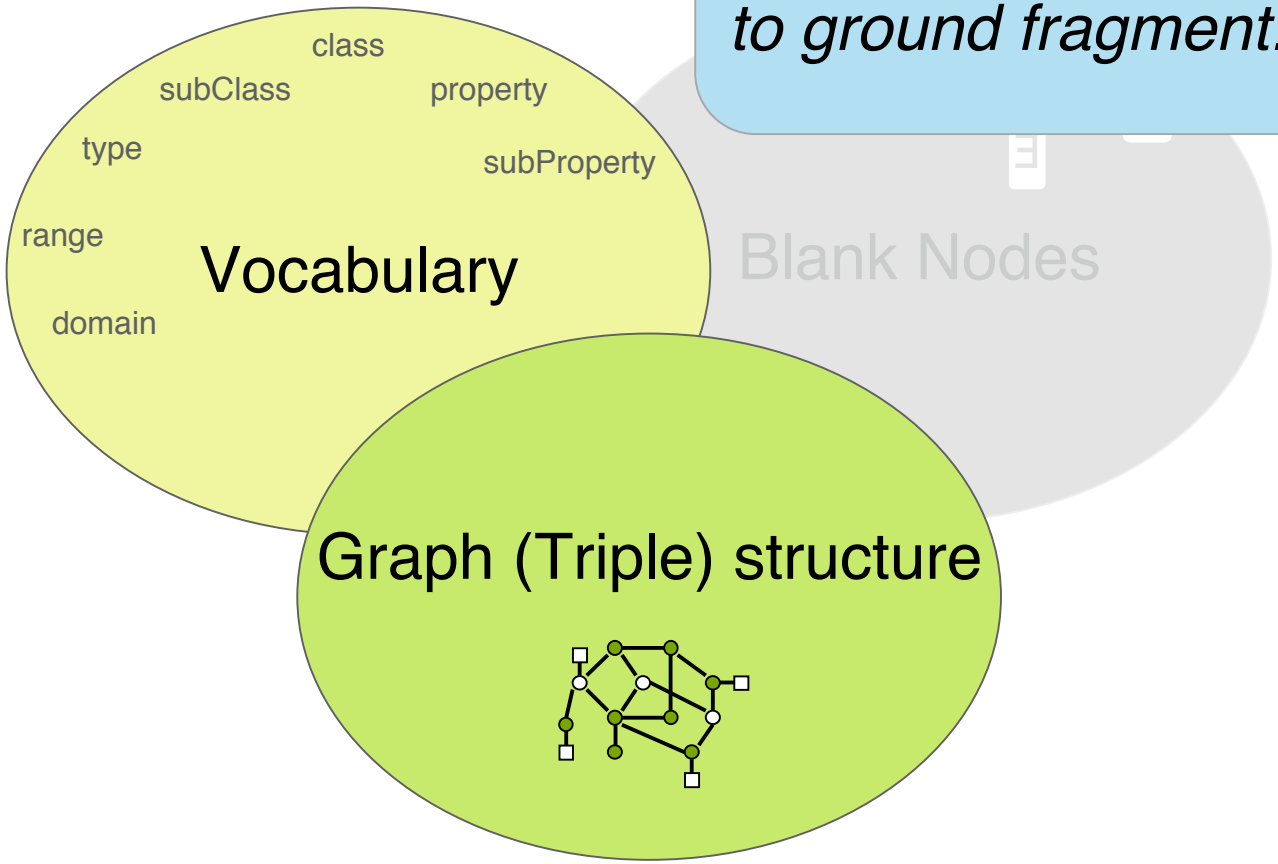


RDF Data Structure: Ground fragment



RDF Data Structure: Ground fragment

Good News: *Blank nodes can be treated orthogonally to ground fragment.*

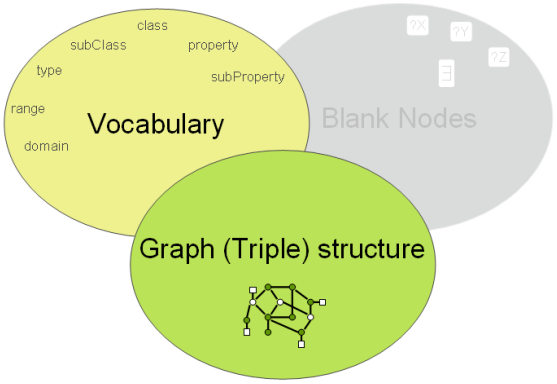


RDF Data Structure: Ground fragment

More good news:

- Vocabulary can be *reduced* to

  { type, domain, range, subClassOf, subPropertyOf }

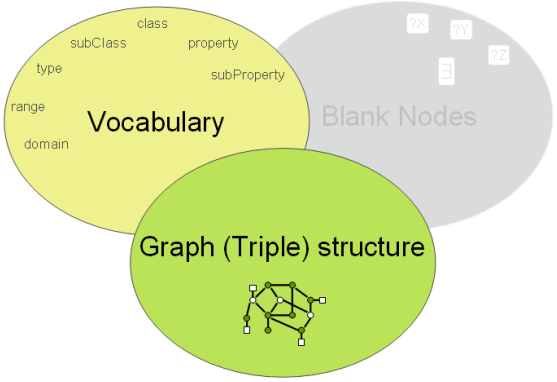


RDF Data Structure: Ground fragment

More good news:

- Vocabulary can be *reduced* to

  { type, domain, range, subClassOf, subPropertyOf }
- Complex semantic rules and axioms can be *avoided*



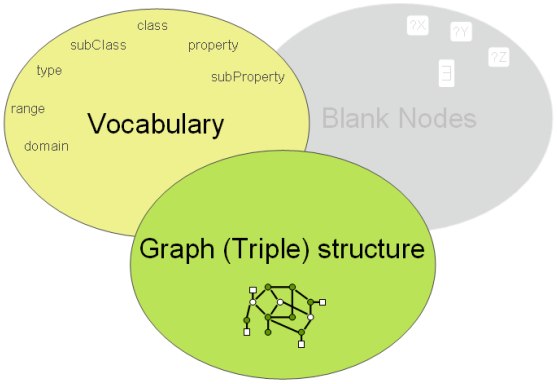
RDF Data Structure: Ground fragment

More good news:

- Vocabulary can be *reduced* to

  { type, domain, range, subClassOf, subPropertyOf }
- Complex semantic rules and axioms can be *avoided*
- Structural (internal) constraints of the language can be *separated* from user-features.

  e.g. (Class, type, Resource)



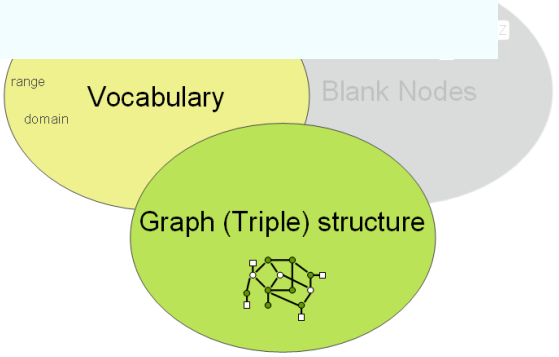
RDF Data Structure: Ground fragment

More good news:

- Vocabulary can be *reduced* to

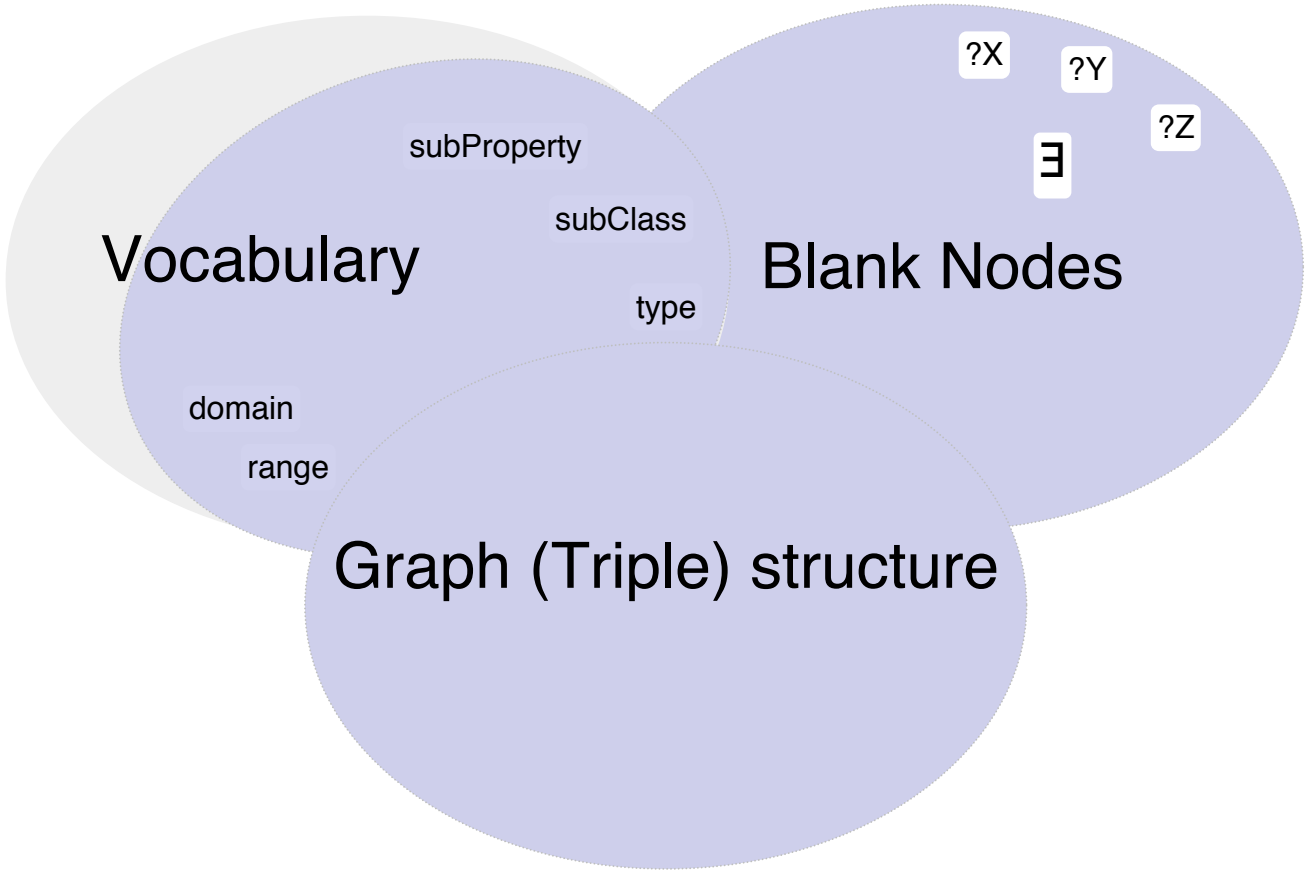
  { type, domain, range, subClassOf, subPropertyOf }
- Complex semantic rules and axioms can be *avoided*
- Structural (internal) constraints of the language can be *separated* from user-features.

  e.g. (Class, type, Resource)
- Features which do not add expressive power can be *avoided*, e.g. reflexivity of subClass and subProperty.



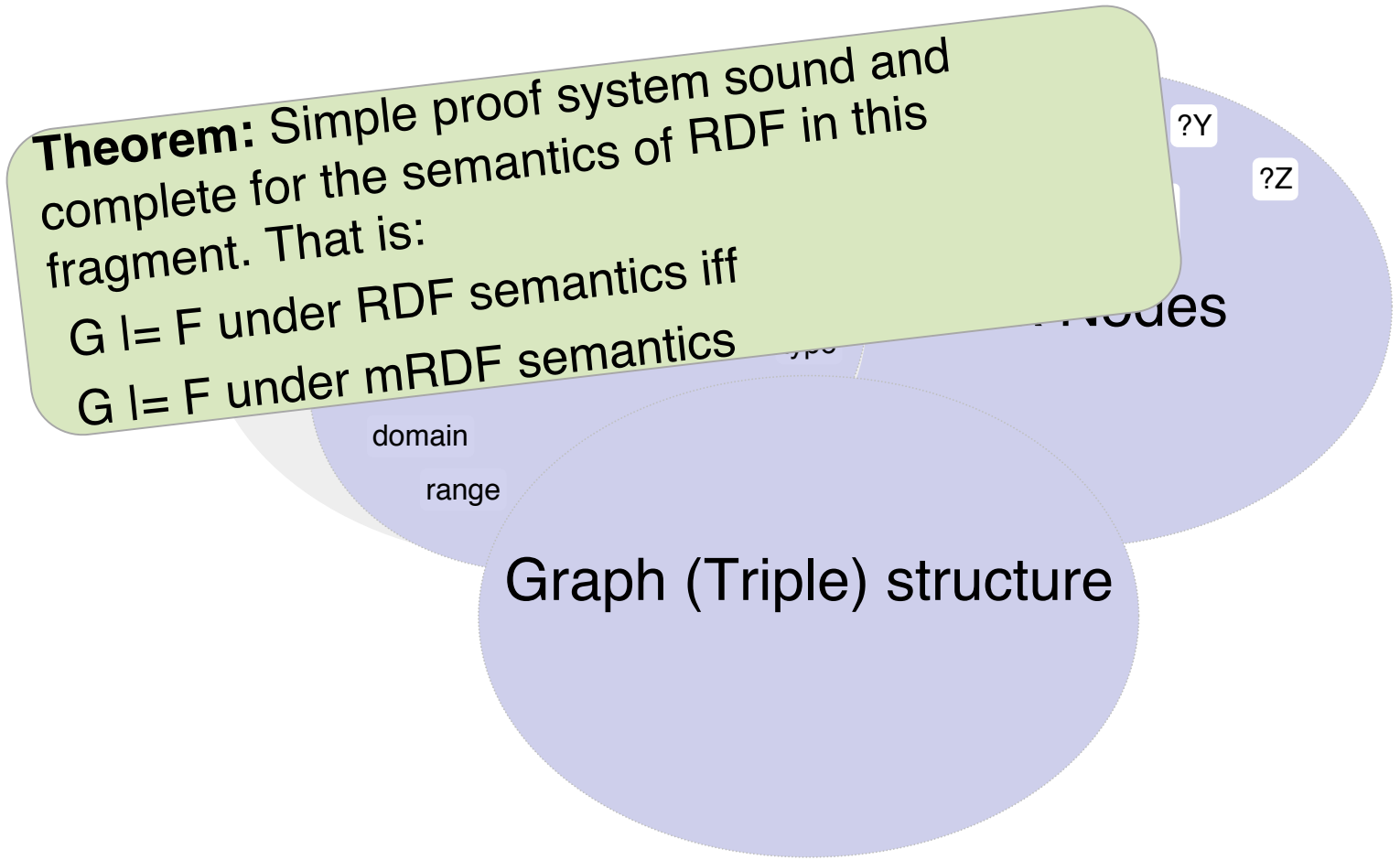
RDF Data Structure: A *minimal fragment*

{subClass, subProperty, type, domain, range}



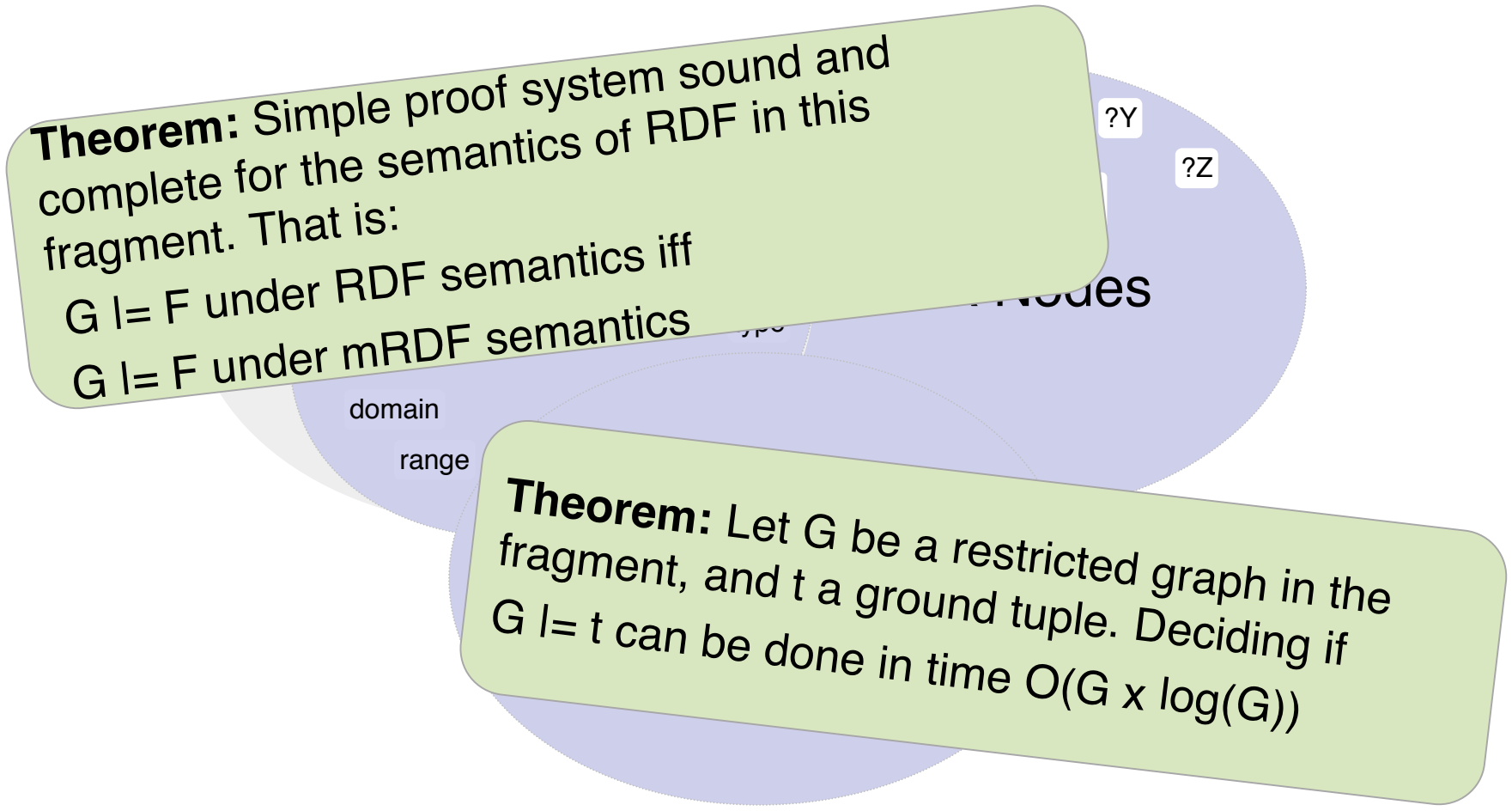
RDF Data Structure: A *minimal fragment*

{subClass, subProperty, type, domain, range}



RDF Data Structure: A *minimal fragment*

{subClass, subProperty, type, domain, range}



Theorem: Simple proof system sound and complete for the semantics of RDF in this fragment. That is:

G |= F under RDF semantics iff

G |= F under mRDF semantics

Theorem: Let G be a restricted graph in the fragment, and t a ground tuple. Deciding if G |= t can be done in time O(G x log(G))

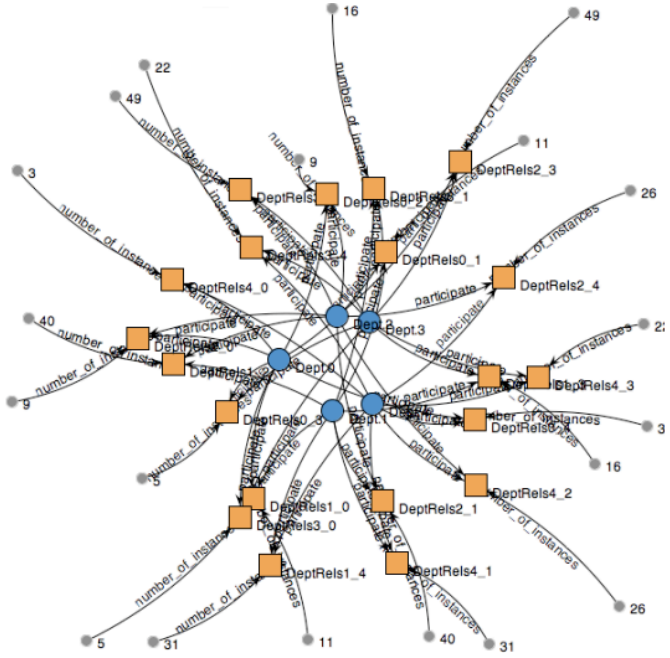
Agenda

1. RDF and Databases
2. RDF and Database models
3. RDF Query Language
   - Requirements and Domains
   - Manifold Views
4. SPARQL
   - Syntax and Semantics
   - Complexity
   - Expressive Power

RDF Query language: Social Networks domain

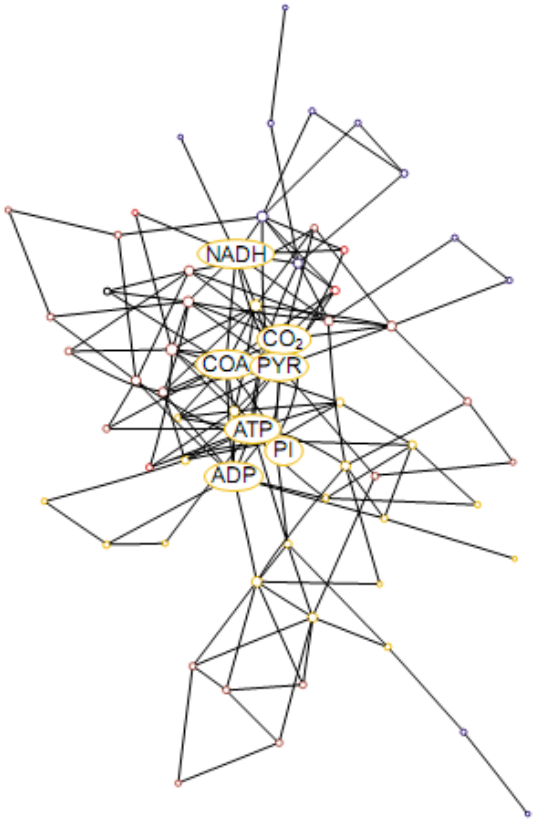
Chapter title	Use Case (local)
Looking for Social Structure	+ Directed to undirected binary relations + Remove relations
Attributes and Relations	+ Extract a subnetwork based on attributes + Group actors based on attributes + Selective grouping of actors based on attributes
Cohesive Subgroups	+ Extract the subnetwork induced by cliques of size n + Build a hierarchy of cliques
Frienship	+ Extract subnetwork by time
Affiliations	+ Two-mode network to one-mode network
Center and Periphery	+ Group multiple binary relations
Brokers and Bridges	+ Extract egonetwork of an actor + Remove relations between groups
Diffusion	+ Selective counting of neighbors + Operations between attributes + Change relation direction based on attributes
Prestige	+ Discretize an attribute
Ranking	+ Find triads by type
Genealogies and Citations	+ Loop removal

Subgraph family	Use Case (Global)
Paths and Cycles	+ Geodesics
Groups (k-neighbors, k-core, n-cliques, k-plex, etc.)	+ Detect cohesive subgroups + Egonetworks + Input Domain
Connected components	+ Connected components + Clustering + Bicomponents and brockers



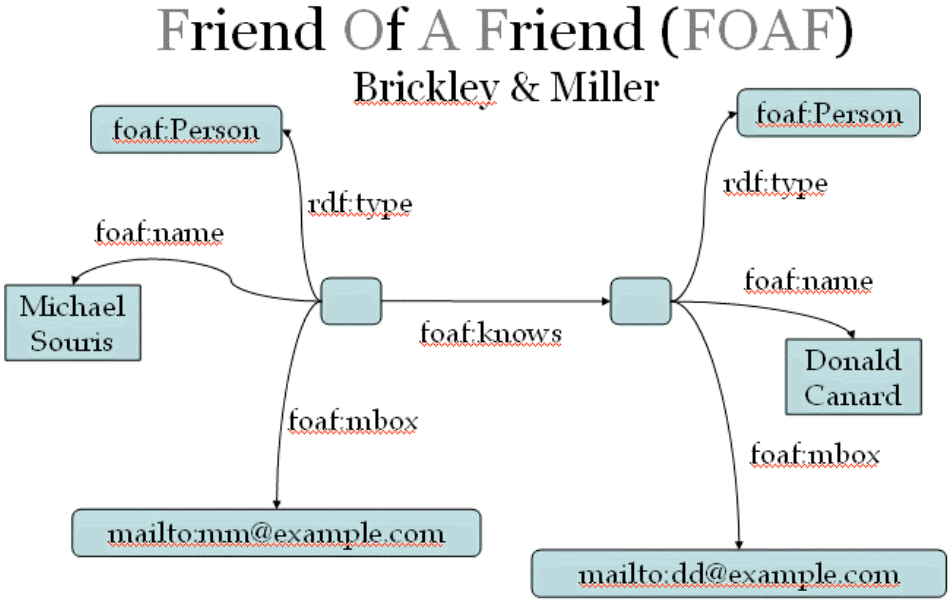
RDF Query Language: Biology domain

Use Case	Graph Query
Chemical structure associated with a node	Node matching
Find the difference in metabolisms between two microbes	Graph intersection, union, difference
To combine multiple protein interaction graphs	Majority graph query
To construct pathways from individual reactions	Graph composition
To connect pathways, metabolism of co-existing organisms	Graph composition
Identify “important” paths from nutrients to chemical outputs	Shortest path queries
Find all products ultimately derived from a particular reaction	Transitive Closure
Observe multiple products are co-regulated	Least common ancestor
To find biopathways graph motifs	Frequent subgraph recognition
Chemical info retrieval	Subgraph isomorphism
Kinaze enzyme	Subgraph homomorphism
Enzyme taxonomies	Subsumption testing
To find biopathways graph motifs	Frequent subgraph recognition



RDF Query Language: Web domain

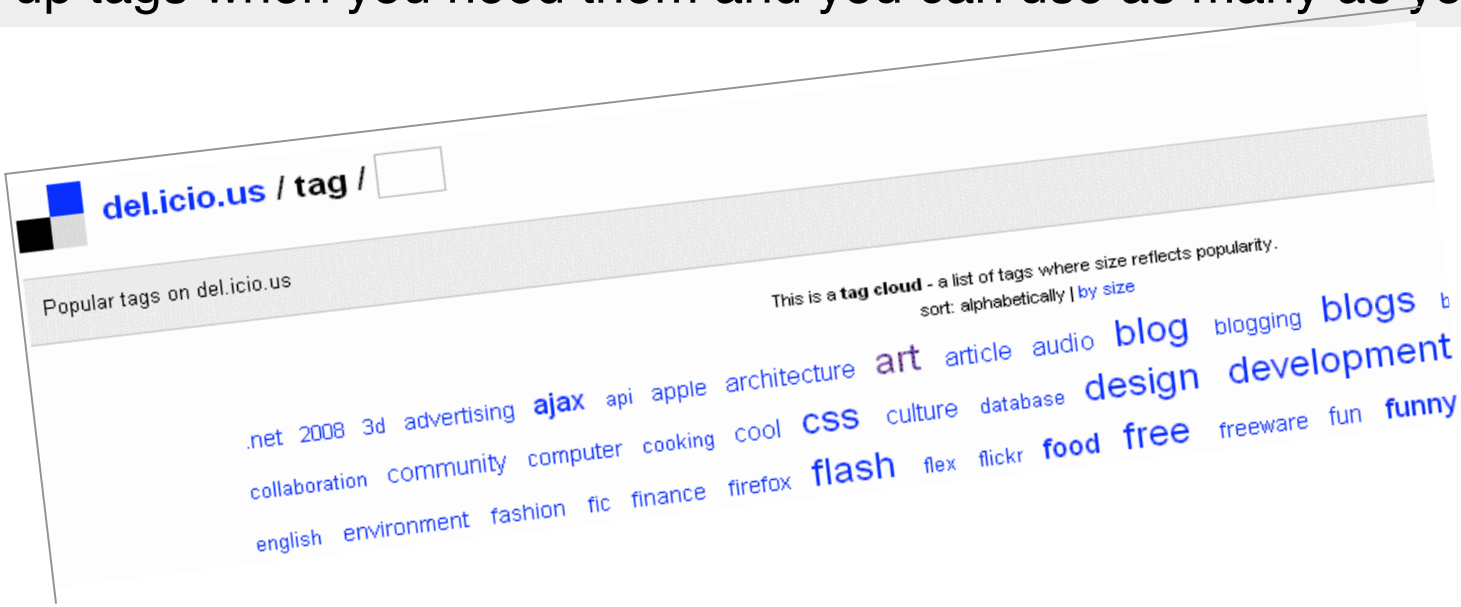
Use Case	Graph Query
What is/are the most cited paper/s?	Degree of a node
What is the influence of article D?	Paths
What is the Erdös distance between authos X and author Y?	Distance
Are suspects A and B related?	Paths
All relatives of degree one of Alice	Adjacency



RDF Query Language: Tagging domain

Tags

A tag is simply a word you use to describe a bookmark. Unlike folders, you make up tags when you need them and you can use as many as you like.



Minimalist design:

- Tags + Bundles (classes)
- No inheritance, no intersection, etc.
- Renaming

RDF Query Language: Standardization's view

- **SQL:** Great for finding data from tabular representations, can get complex when many tables are involved in a given query

SQL, XQuery and SPARQL: What's Wrong with this Picture?

Jim Melton (Oracle; XML Query Working Group, XML Coord. Group)

Sixth annual W3C Technical Plenary (March 2006)

RDF Query Language: Standardization's view

- **SQL:** Great for finding data from tabular representations, can get complex when many tables are involved in a given query
- **XQuery:** Great for finding data in tree representations, can get complex when many relationships have to be traversed

SQL, XQuery and SPARQL: What's Wrong with this Picture?

Jim Melton (Oracle; XML Query Working Group, XML Coord. Group)

Sixth annual W3C Technical Plenary (March 2006)

Standardization's view (Jim Melton, Oracle, 2006)

- **SQL:** Great for finding data from tabular representations, can get complex when many tables are involved in a given query
- **XQuery:** Great for finding data in tree representations, can get complex when many relationships have to be traversed
- **SPARQL**: Good pattern matching paradigm, especially when relationships have to be used to answer a query

SQL, XQuery and SPARQL: What's Wrong with this Picture?

Jim Melton (Oracle; XML Query Working Group, XML Coord. Group)

Sixth annual W3C Technical Plenary (March 2006)

Standardization's view (Jim Melton, Oracle, 2006)

- **SQL:** Great for finding data from tabular representations, can get complex when many tables are involved in a given query
- **XQuery:** Great for finding data in tree representations, can get complex when many relationships have to be traversed
- **SPARQL**: Good pattern m… …digm, especially when relationsh… …be used to answer a query

SPARQL = Sympathy Queen?

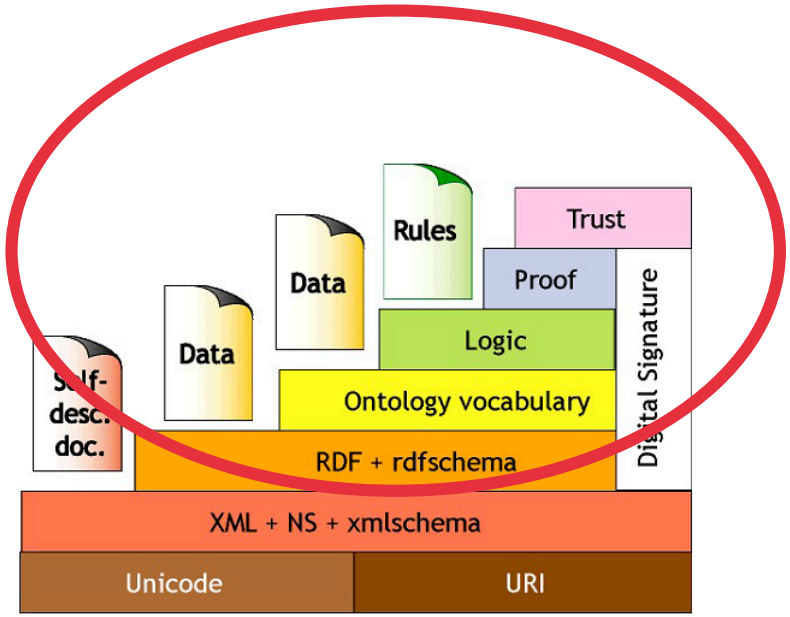
SQL, XQuery and SPARQL: What's Wrong with this Picture?

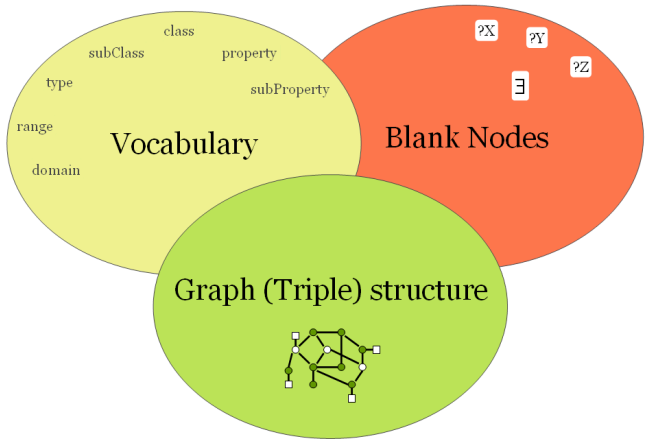
Jim Melton (Oracle; XML Query Working Group, XML Coord. Group)

Sixth annual W3C Technical Plenary (March 2006)

RDF Query Language: Logician's view

- RDF is the first level of a logical tower
- Emphasis in logic features of RDF model
- Keep an eye in extensions to more expressive logics
- Bad news: complexity issues

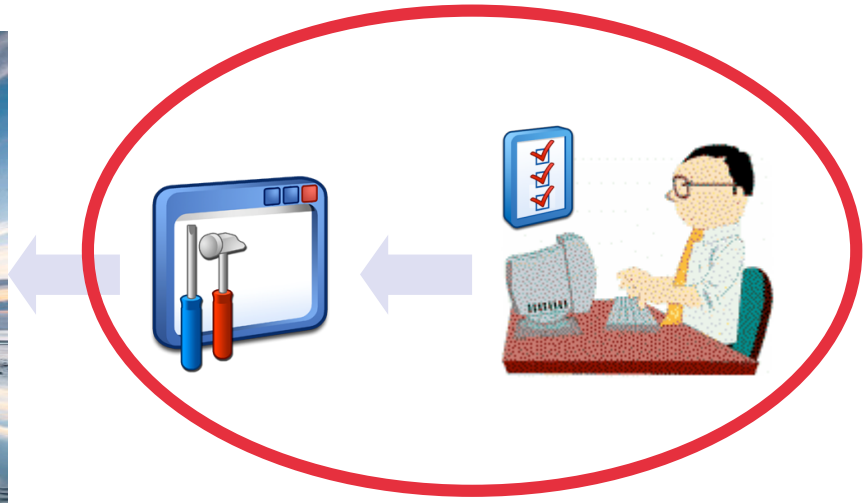




RDF Query Language: Developer's view

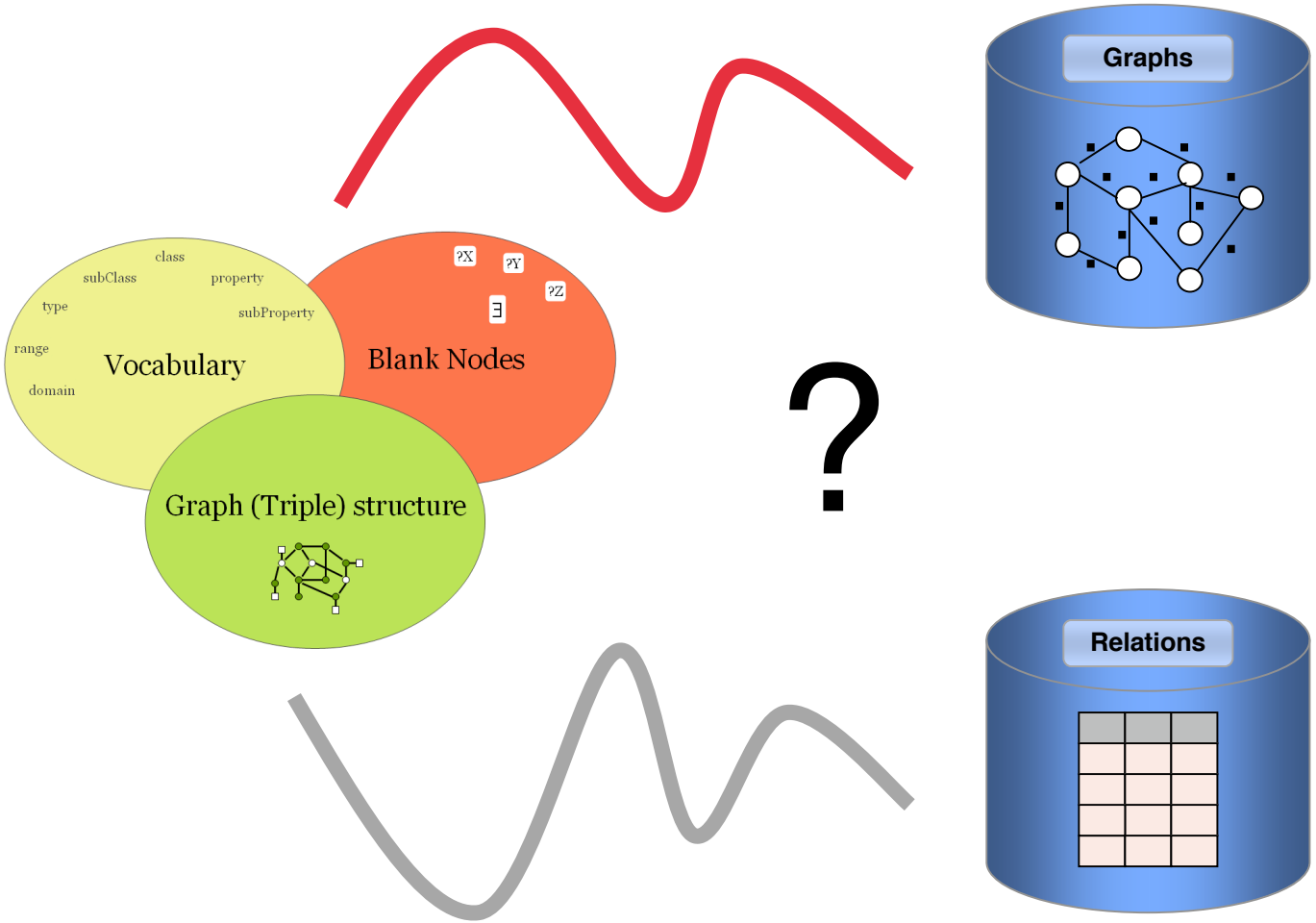
- How do we answer the most common queries?
- How do we cope with APIs and store developments?
- Design usually influenced by current programming and system tools.
- Not always concerned with scalability and long term.





RDF Query Language: Database theoretician's view

RDF as a graph data model?



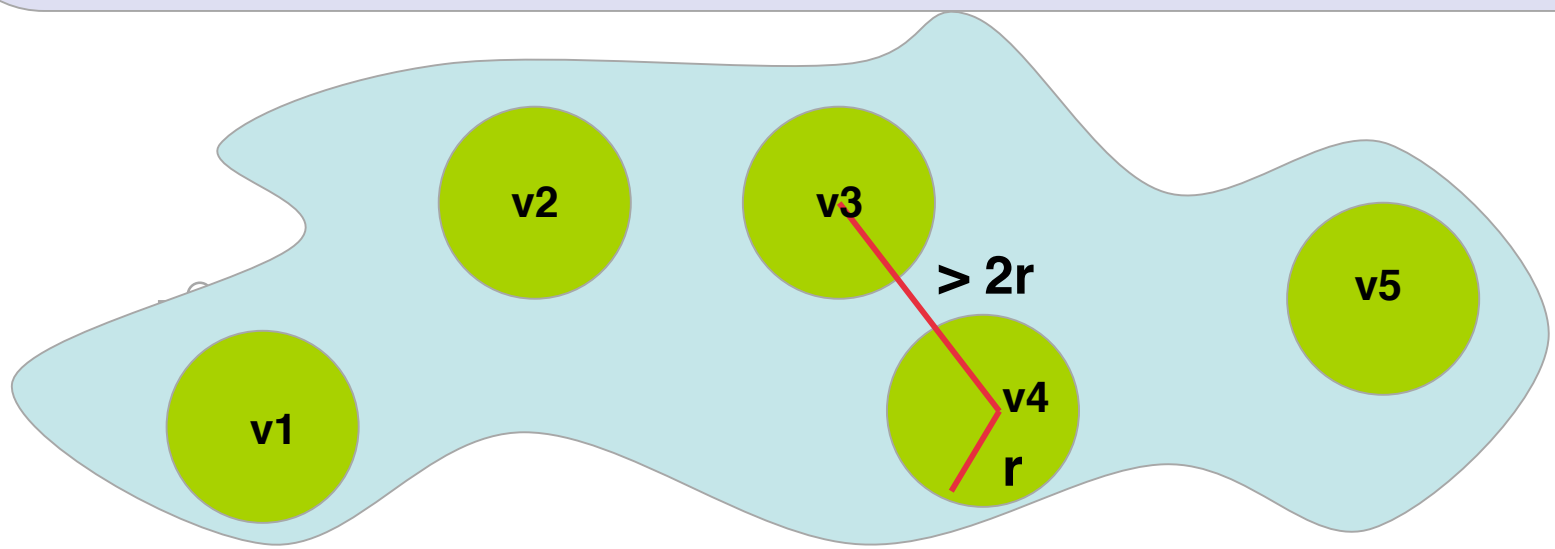
RDF as a relational model?

RDF Query Language: Database theoretician's view

Theorem (Gaifman). A property of graphs is expressible by a closed first order formula iff it is equivalent to a combination of properties of the form

$$\exists v_1, \ldots, v_s \Big[\bigwedge_{1 \leq i \leq s} P(N(v_i, r)) \wedge \bigwedge_{1 \leq i < j \leq s} d(v_i, v_j) > 2r \Big]$$

where $v_1,\ldots,v_s$ denote vertices and d(x,y) denotes distance



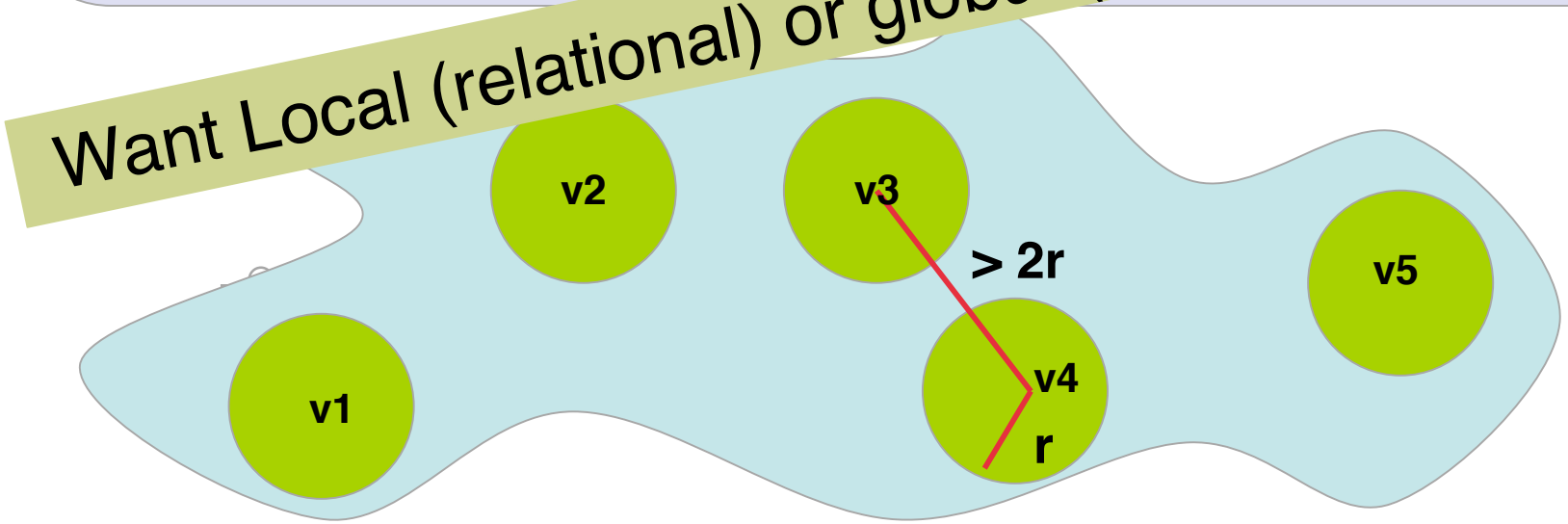
RDF Query Language: Database theoretician's view

Theorem (Gaifman). A property of graphs is expressible by a closed first order formula iff it is equivalent to a combination of properties of the form

$$\exists v_1, \ldots, v_s \Big[\bigwedge_{1 \leq i \leq s} P(N(v_i, r)) \wedge \bigwedge_{1 \leq i < j \leq s} d(v_i, v_j) > 2r \Big]$$

where $v_1,\ldots,v_s$ denote vertices and d(x,y)

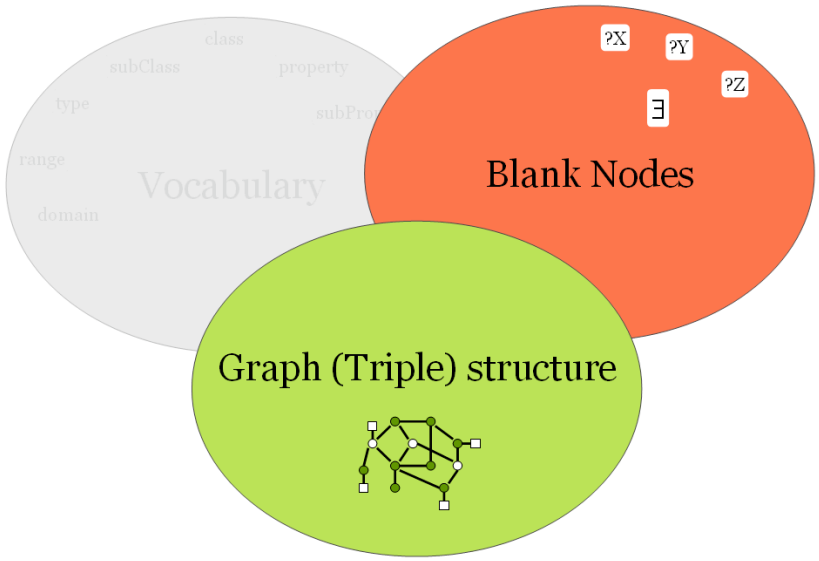
Want Local (relational) or global (graph) queries?



W3C Working Group's view

SPARQL (W3C Recommendation, 2008)

- Relational view of querying
- RDF = triples + blanks
- Pattern matching

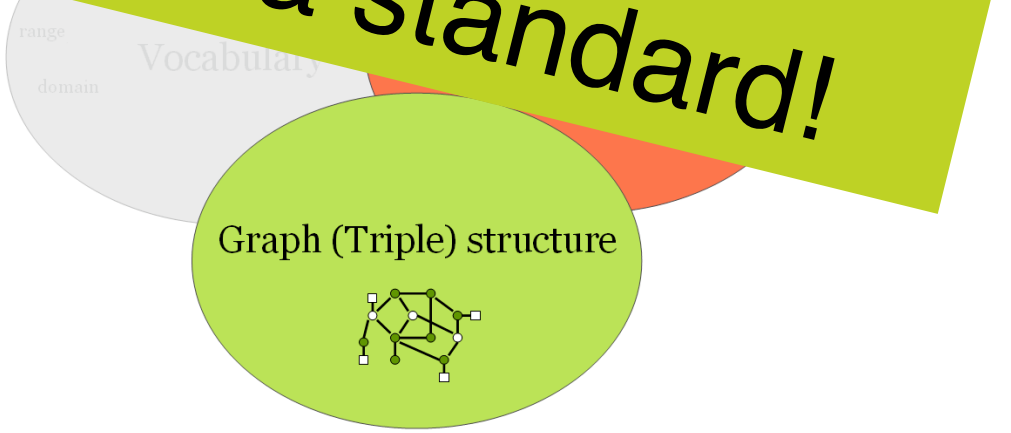


W3C Working Group's view

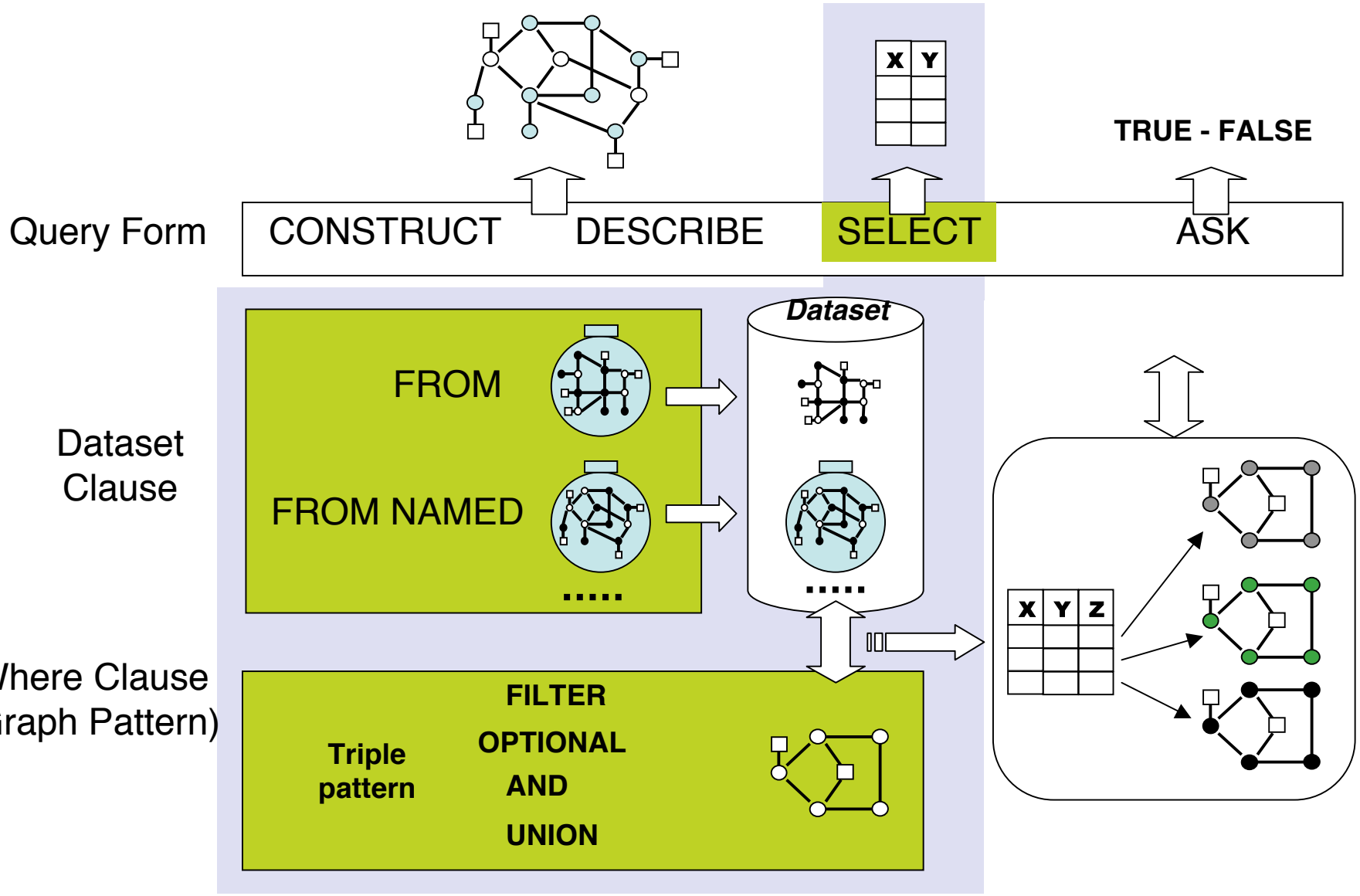
SPARQL (W3C Recommendation, 2008)

- Re[illegible]tional view of querying
- R[illegible]anks
- F[illegible]

Good News:
there is a standard!



SPARQL Query (General Structure)



Outline

- Overview of syntax and semantics of SPARQL
- Formal semantics of SPARQL
- Complexity of the SPARQL evaluation problem
- Expressive Power of SPARQL

Core of the language: Example

```
SELECT ?Name ?Email
WHERE
{
  ?X :name ?Name
  ?X :email ?Email
}
```

Core of the language: Example

```
SELECT ?Name ?Email
WHERE
{
  ?X :name ?Name
  ?X :email ?Email
}
```

Core of the language: Example

```
SELECT ?Name ?Email
WHERE
{
  ?X :name ?Name
  ?X :email ?Email
}
```

Core of the language: Example

```
SELECT ?Name ?Email
WHERE
{
  ?X :name ?Name
  ?X :email ?Email
}
```

Core of the language: Example

```
SELECT ?Name ?Email
WHERE
{
  ?X :name ?Name
  ?X :email ?Email
}
```

In general, in a query we have:

$$H \leftarrow$$

- Head: processing of some variables.

Core of the language: Example

```
SELECT ?Name ?Email
WHERE
{
  ?X :name ?Name
  ?X :email ?Email
}
```

In general, in a query we have:

$$H \leftarrow P$$

- Head: processing of some variables.
- Body: pattern matching expression.

Core of the language: Example

```
SELECT ?Name ?Email
WHERE
{
  ?X :name ?Name
  ?X :email ?Email
}
```

In general, in a query we have:

$$H \leftarrow P$$

- Head: processing of some variables.
- Body: pattern matching expression.

We focus on P.

But things can become more complex ...

Interesting features of pattern matching on graphs

- Grouping
- Optional parts
- Nesting
- Union of patterns
- Filtering
- ...

```
{ P1
  P2 }
```

But things can become more complex ...

Interesting features of pattern matching on graphs

- Grouping
- Optional parts
- Nesting
- Union of patterns
- Filtering
- ...

```
{ { P1
    P2 }

  { P3
    P4 }

}
```

But things can become more complex ...

Interesting features of pattern matching on graphs

- Grouping
- Optional parts
- Nesting
- Union of patterns
- Filtering
- ...

```
{ { P1
    P2
    OPTIONAL { P5 }  }

  { P3
    P4
    OPTIONAL { P7 }  }

}
```

But things can become more complex ...

Interesting features of pattern matching on graphs

- Grouping
- Optional parts
- Nesting
- Union of patterns
- Filtering
- ...

```
{ { P1
    P2
    OPTIONAL { P5 }  }

  { P3
    P4
    OPTIONAL { P7
      OPTIONAL { P8 }  }  }
}
```

But things can become more complex ...

Interesting features of pattern matching on graphs

- Grouping
- Optional parts
- Nesting
- Union of patterns
- Filtering
- ...

```
{ { P1
    P2
    OPTIONAL { P5 }  }

  { P3
    P4
    OPTIONAL { P7
      OPTIONAL { P8 }  }  }
}
UNION
{ P9 }
```

But things can become more complex ...

Interesting features of pattern matching on graphs

- Grouping
- Optional parts
- Nesting
- Union of patterns
- Filtering
- ...

```
{ { P1
    P2
    OPTIONAL { P5 }  }

  { P3
    P4
    OPTIONAL { P7
      OPTIONAL { P8 }  }  }
}
UNION
{ P9
  FILTER ( R ) }
```

But things can become more complex ...

Interesting features of pattern matching on graphs

- Grouping
- Optional parts
- Nesting
- Union of patterns
- Filtering
- ...

```
{ { P1
    P2
    OPTIONAL { P5 }  }

  { P3
    P4
    OPTIONAL { P7
      OPTIONAL { P8 }  }  }
}
UNION
{ P9
  FILTER ( R ) }
```

A standard algebraic syntax

- Triple patterns: RDF triples + variables

`?X :name "john"`	$(?X, \text{name}, \text{john})$

- Graph patterns: full parenthesized algebra

`{ P1 P2 }`	$(P_1 \text{ AND } P_2)$
`{ P1 OPTIONAL { P2 }}`	$(P_1 \text{ OPT } P_2)$
`{ P1 } UNION { P2 }`	$(P_1 \text{ UNION } P_2)$
`{ P1 FILTER ( R ) }`	$(P_1 \text{ FILTER } R)$
original SPARQL syntax	algebraic syntax

A formal semantics for SPARQL

A formal approach is beneficial for:

- Providing the user an ultimate guide of language behavior
- Clarifying and expliciting corner cases
- Helping and simplifying the implementation process
- Providing sound foundations

A formal semantics for SPARQL

Desiderata for semantics:

A formal semantics for SPARQL

Desiderata for semantics:

- Compositional approach: The meaning of an expression is determined by the meaning of its parts and the way they are combined.

A formal semantics for SPARQL

Desiderata for semantics:

- Compositional approach: The meaning of an expression is determined by the meaning of its parts and the way they are combined.
- Denotational approach: Meaning of expressions is formalized by assigning mathematical objects which describe the meaning.

A formal semantics for SPARQL

Desiderata for semantics:

- Compositional approach: The meaning of an expression is determined by the meaning of its parts and the way they are combined.
- Denotational approach: Meaning of expressions is formalized by assigning mathematical objects which describe the meaning.

Will present:

- A denotational and compositional semantics.
- A comparison of it with W3C Semantics of SPARQL

Mappings: building block for the semantics

Definition

A mapping is a partial function from variables to RDF terms.

Mappings: building block for the semantics

Definition

A mapping is a partial function from variables to RDF terms.

The evaluation of a pattern results in a set of mappings.

Mappings: building block for the semantics

Definition

A mapping is a partial function from variables to RDF terms.

The evaluation of a pattern results in a set of mappings.

Example (Relational view)

- Variables $\mapsto$ Attributes
- Mappings $\mapsto$ Tuples
- Set of mappings $\mapsto$ Tables

The semantics of triple patterns

Given an RDF graph and a triple pattern t

Definition

The evaluation of t is the set of mappings that

The semantics of triple patterns

Given an RDF graph and a triple pattern t

Definition

The evaluation of t is the set of mappings that

- make t to match the graph

The semantics of triple patterns

Given an RDF graph and a triple pattern t

Definition

The evaluation of t is the set of mappings that

- make t to match the graph
- have as domain the variables in t.

The semantics of triple patterns

Given an RDF graph and a triple pattern t

Definition

The evaluation of t is the set of mappings that

- make t to match the graph
- have as domain the variables in t.

Example

graph

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

triple

$(?X, \text{name}, ?Y)$

evaluation

	$?X$	$?Y$
μ_1:	R_1	john
μ_2:	R_2	paul

The semantics of triple patterns

Given an RDF graph and a triple pattern t

Definition

The evaluation of t is the set of mappings that

- make t to match the graph
- have as domain the variables in t.

Example

graph

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

triple

$(?X, \text{name}, ?Y)$

evaluation

	$?X$	$?Y$
μ_1:	R_1	john
μ_2:	R_2	paul

The semantics of triple patterns

Given an RDF graph and a triple pattern t

Definition

The evaluation of t is the set of mappings that

- make t to match the graph
- have as domain the variables in t.

Example

graph

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

triple

$(?X, \text{name}, ?Y)$

evaluation

	$?X$	$?Y$
μ_1:	R_1	john
μ_2:	R_2	paul

The bag semantics of triple patterns

Definition (Bag Semantics)

The evaluation of t is the multisetset (bag) of mappings that

- make t to match the graph
- have as domain the variables in t.

The bag semantics of triple patterns

Definition (Bag Semantics)

The evaluation of t is the multisetset (bag) of mappings that

- make t to match the graph
- have as domain the variables in t.

Bag Semantics

- Reflects real world practice

The bag semantics of triple patterns

Definition (Bag Semantics)

The evaluation of t is the multisetset (bag) of mappings that

- make t to match the graph
- have as domain the variables in t.

Bag Semantics

- Reflects real world practice
- Not well understood from a theoretical point of view

The bag semantics of triple patterns

Definition (Bag Semantics)

The evaluation of t is the multisetset (bag) of mappings that

- make t to match the graph
- have as domain the variables in t.

Bag Semantics

- Reflects real world practice
- Not well understood from a theoretical point of view
- For RDF/SPARQL, really set/bag semantics (sets for data input, bag for subsequent processing and output).

The bag semantics of triple patterns

Definition (Bag Semantics)

The evaluation of t is the multisetset (bag) of mappings that

- make t to match the graph
- have as domain the variables in t.

Bag Semantics

- Reflects real world practice
- Not well understood from a theoretical point of view
- For RDF/SPARQL, really set/bag semantics (sets for data input, bag for subsequent processing and output).

This talk will avoid bag semantics details.

Compatible mappings

Definition

Two mappings are compatible if they agree in their shared variables.

Example

	?X	?Y	?Z	?V
μ_1 :	R_1	john		
μ_2 :	R_1		J@edu.ex	
μ_3 :			P@edu.ex	R_2

Compatible mappings

Definition

Two mappings are compatible if they agree in their shared variables.

Example

	?X	?Y	?Z	?V
μ_1 :	R_1	john		
μ_2 :	R_1		J@edu.ex	
μ_3 :			P@edu.ex	R_2

Compatible mappings

Definition

Two mappings are compatible if they agree in their shared variables.

Example

	?X	?Y	?Z	?V
μ_1 :	R_1	john		
μ_2 :	R_1		J@edu.ex	
μ_3 :			P@edu.ex	R_2
$\mu_1 \cup \mu_2$:	R_1	john	J@edu.ex	

Compatible mappings

Definition

Two mappings are compatible if they agree in their shared variables.

Example

	?X	?Y	?Z	?V
μ_1 :	R_1	john		
μ_2 :	R_1		J@edu.ex	
μ_3 :			P@edu.ex	R_2
$\mu_1 \cup \mu_2$:	R_1	john	J@edu.ex	

Compatible mappings

Definition

Two mappings are compatible if they agree in their shared variables.

Example

	?X	?Y	?Z	?V
μ_1 :	R_1	john		
μ_2 :	R_1		J@edu.ex	
μ_3 :			P@edu.ex	R_2
$\mu_1 \cup \mu_2$:	R_1	john	J@edu.ex	
$\mu_1 \cup \mu_3$:	R_1	john	P@edu.ex	R_2

Compatible mappings

Definition

Two mappings are compatible if they agree in their shared variables.

Example

	?X	?Y	?Z	?V
μ_1 :	R_1	john		
μ_2 :	R_1		J@edu.ex	
μ_3 :			P@edu.ex	R_2
$\mu_1 \cup \mu_2$:	R_1	john	J@edu.ex	
$\mu_1 \cup \mu_3$:	R_1	john	P@edu.ex	R_2

- μ_2 and μ_3 are not compatible

Sets of mappings and operations

Let M_1 and M_2 be sets of mappings:

Definition

Sets of mappings and operations

Let M_1 and M_2 be sets of mappings:

Definition

Join: $M_1 \bowtie M_2$

- extending mappings in M_1 with compatible mappings in M_2

Sets of mappings and operations

Let M_1 and M_2 be sets of mappings:

Definition

Join: $M_1 \bowtie M_2$

- extending mappings in M_1 with compatible mappings in M_2

Difference: $M_1 \setminus M_2$

- mappings in M_1 that cannot be extended with mappings in M_2

Sets of mappings and operations

Let M_1 and M_2 be sets of mappings:

Definition

Join: $M_1 \bowtie M_2$

- extending mappings in M_1 with compatible mappings in M_2

Difference: $M_1 \setminus M_2$

- mappings in M_1 that cannot be extended with mappings in M_2

Union: $M_1 \cup M_2$

- mappings in M_1 plus mappings in M_2 (set theoretical union)

Sets of mappings and operations

Let M_1 and M_2 be sets of mappings:

Definition

Join: $M_1 \bowtie M_2$

- extending mappings in M_1 with compatible mappings in M_2

Difference: $M_1 \setminus M_2$

- mappings in M_1 that cannot be extended with mappings in M_2

Union: $M_1 \cup M_2$

- mappings in M_1 plus mappings in M_2 (set theoretical union)

Definition

Left Outer Join: $M_1 \mathbin{⟕} M_2 = (M_1 \bowtie M_2) \cup (M_1 \setminus M_2)$

Semantics of SPARQL operators

Compositional semantics at work:

Definition

Given P_1, P_2 graph patterns and D an RDF graph:

$$[\![P_1 \text{ AND } P_2]\!]_D \quad \rightarrow$$

$$[\![P_1 \text{ UNION } P_2]\!]_D \quad \rightarrow$$

$$[\![P_1 \text{ OPT } P_2]\!]_D \quad \rightarrow$$

Semantics of SPARQL operators

Compositional semantics at work:

Definition

Given P_1, P_2 graph patterns and D an RDF graph:

$$[\![P_1 \text{ AND } P_2]\!]_D \quad \rightarrow \quad [\![P_1]\!]_D \bowtie [\![P_2]\!]_D$$

$$[\![P_1 \text{ UNION } P_2]\!]_D \quad \rightarrow$$

$$[\![P_1 \text{ OPT } P_2]\!]_D \quad \rightarrow$$

Semantics of SPARQL operators

Compositional semantics at work:

Definition

Given P_1, P_2 graph patterns and D an RDF graph:

$$[\![P_1 \text{ AND } P_2]\!]_D \quad \rightarrow \quad [\![P_1]\!]_D \bowtie [\![P_2]\!]_D$$

$$[\![P_1 \text{ UNION } P_2]\!]_D \quad \rightarrow \quad [\![P_1]\!]_D \cup [\![P_2]\!]_D$$

$$[\![P_1 \text{ OPT } P_2]\!]_D \quad \rightarrow$$

Semantics of SPARQL operators

Compositional semantics at work:

Definition

Given P_1, P_2 graph patterns and D an RDF graph:

$$[\![P_1 \text{ AND } P_2]\!]_D \quad \rightarrow \quad [\![P_1]\!]_D \bowtie [\![P_2]\!]_D$$

$$[\![P_1 \text{ UNION } P_2]\!]_D \quad \rightarrow \quad [\![P_1]\!]_D \cup [\![P_2]\!]_D$$

$$[\![P_1 \text{ OPT } P_2]\!]_D \quad \rightarrow \quad [\![P_1]\!]_D \mathbin{⟕} [\![P_2]\!]_D$$

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$$((?X, \text{name}, ?Y) \text{ OPT } (?X, \text{email}, ?E))$$

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$$((?X, \text{ name}, ?Y) \text{ OPT } (?X, \text{ email}, ?E))$$

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$$((?X, \text{name}, ?Y) \text{ OPT } (?X, \text{email}, ?E))$$

$?X$	$?Y$
R_1	john
R_2	paul

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$((?X, \text{name}, ?Y) \text{ OPT } (?X, \text{email}, ?E))$

$?X$	$?Y$
R_1	john
R_2	paul

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$((?X, \text{name}, ?Y) \text{ OPT } (?X, \text{email}, ?E))$

$?X$	$?Y$
R_1	john
R_2	paul

$?X$	$?E$
R_1	J@ed.ex

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$((?X, \text{name}, ?Y) \text{ OPT } (?X, \text{email}, ?E))$

$?X$	$?Y$
R_1	john
R_2	paul

$?X$	$?E$
R_1	J@ed.ex

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$((?X, \text{name}, ?Y) \text{ OPT } (?X, \text{email}, ?E))$

$?X$	$?Y$
R_1	john
R_2	paul

$?X$	$?Y$	$?E$
R_1	john	J@ed.ex
R_2	paul	

$?X$	$?E$
R_1	J@ed.ex

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$((?X, \text{name}, ?Y) \text{ OPT } (?X, \text{email}, ?E))$

$?X$	$?Y$
R_1	john
R_2	paul

$?X$	$?Y$	$?E$
R_1	john	J@ed.ex
R_2	paul	

$?X$	$?E$
R_1	J@ed.ex

- from the Join

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$((?X, \text{name}, ?Y) \text{ OPT } (?X, \text{email}, ?E))$

$?X$	$?Y$
R_1	john
R_2	paul

$?X$	$?Y$	$?E$
R_1	john	J@ed.ex
R_2	paul	

$?X$	$?E$
R_1	J@ed.ex

- from the Difference

Simple example

Example

$(R_1, \text{name}, \text{john})$
$(R_1, \text{email}, \text{J@ed.ex})$
$(R_2, \text{name}, \text{paul})$

$((?X, \text{name}, ?Y) \text{ OPT } (?X, \text{email}, ?E))$

$?X$	$?Y$
R_1	john
R_2	paul

$?X$	$?Y$	$?E$
R_1	john	J@ed.ex
R_2	paul	

$?X$	$?E$
R_1	J@ed.ex

- from the Union

Semantics of FILTER patterns

In a pattern (P FILTER F), the filter expression F is a Boolean combination of atoms.

Semantics of FILTER patterns

In a pattern (P FILTER F), the filter expression F is a Boolean combination of atoms.

A mapping satisfies an atom:

- $(?X = c)$ if it gives the value c to variable $?X$
- $(?X = ?Y)$ if it gives the same value to $?X$ and $?Y$
- bound$(?X)$ if it is defined for $?X$

Semantics of FILTER patterns

In a pattern (P FILTER F), the filter expression F is a Boolean combination of atoms.

A mapping satisfies an atom:

- $(?X = c)$ if it gives the value c to variable $?X$
- $(?X = ?Y)$ if it gives the same value to $?X$ and $?Y$
- bound$(?X)$ if it is defined for $?X$

Definition

$$[\![P \text{ FILTER } R]\!] = \{ \mu \in [\![P]\!] : \mu \models R\}$$
$$= \text{Set of mappings in } [\![P]\!] \text{ that satisfy } R.$$

Semantics of FILTER patterns

In a pattern (P FILTER F), the filter expression F is a Boolean combination of atoms.

A mapping satisfies an atom:

- $(?X = c)$ if it gives the value c to variable $?X$
- $(?X = ?Y)$ if it gives the same value to $?X$ and $?Y$
- bound$(?X)$ if it is defined for $?X$

Definition

$$[\![P \text{ FILTER } R]\!] = \{ \mu \in [\![P]\!] : \mu \models R\}$$
$$= \text{Set of mappings in } [\![P]\!] \text{ that satisfy } R.$$

Makes sense only if $\text{var}(R) \subseteq \text{var}(P)$ (safe filters).

Complexity (The evaluation problem)

Input:

mapping μ,
graph pattern P ,
RDF graph D.

Question:

Is the mapping in the evaluation of the pattern against the graph?
Formally:

$$\text{Is it true that } \mu \in [\![P]\!]_D \ ?$$

Evaluation of simple patterns is polynomial.

Theorem

For patterns using only AND *and* FILTER *operators, the evaluation problem is polynomial:*

$$O(\textit{size of the pattern} \times \textit{size of the graph}).$$

Evaluation of simple patterns is polynomial.

Theorem

For patterns using only AND *and* FILTER *operators, the evaluation problem is polynomial:*

$$O(\textit{size of the pattern} \times \textit{size of the graph}).$$

Proof idea

- *Check that the mapping makes every triple to match.*
- *Then check that the mapping satisfies the* FILTER*s.*

Evaluation including UNION is NP-complete.

Theorem

For patterns using only AND, FILTER *and* UNION *operators, the evaluation problem is NP-complete.*

Evaluation including UNION is NP-complete.

Theorem

For patterns using only AND, FILTER *and* UNION *operators, the evaluation problem is NP-complete.*

Proof idea

- *Reduction from 3SAT.*
- *A pattern encodes the propositional formula.*
- $\neg$ bound *is used to encode negation.*

Evaluation including UNION is NP-complete.

Theorem

For patterns using only AND, FILTER *and* UNION *operators, the evaluation problem is NP-complete.*

Proof idea

- *Reduction from 3SAT.*
- *A pattern encodes the propositional formula.*
- $\neg$ bound *is used to encode negation.*

In general: Evaluation problem is PSPACE-complete.

Theorem

For general patterns that include OPT *operator, the evaluation problem is PSPACE-complete.*

In general: Evaluation problem is PSPACE-complete.

Theorem

For general patterns that include OPT *operator, the evaluation problem is PSPACE-complete.*

Proof idea

- *Reduction from QBF*
- *A pattern encodes a quantified propositional formula:*

$$\forall x_1 \exists y_1 \forall x_2 \exists y_2 \cdots \psi.$$

- *nested* OPT*s are used to encode quantifier alternation.*

In general: Evaluation problem is PSPACE-complete.

Theorem

For general patterns that include OPT *operator, the evaluation problem is PSPACE-complete.*

Proof idea

- *Reduction from QBF*
- *A pattern encodes a quantified propositional formula:*

$$\forall x_1 \exists y_1 \forall x_2 \exists y_2 \cdots \psi.$$

- *nested* OPT*s are used to encode quantifier alternation.*

Data–complexity is polynomial

Theorem

When patterns are consider to be fixed (data complexity), the evaluation problem is in LOGSPACE.

Data–complexity is polynomial

Theorem

When patterns are consider to be fixed (data complexity), the evaluation problem is in LOGSPACE.

Proof idea

From data–complexity of first–order logic.

Expressive Power of SPARQL

- A query is a function from the set of input data to the set of output data.
- The expressive power of a query language is given by the set of queries it can express.

Expressive Power of SPARQL

- A query is a function from the set of input data to the set of output data.
- The expressive power of a query language is given by the set of queries it can express.

Definition (Equivalence of languages)

Two query languages L_1 and L_2 have the same expressive power if they can express the same queries.

Expressive Power of SPARQL

- A query is a function from the set of input data to the set of output data.
- The expressive power of a query language is given by the set of queries it can express.

Definition (Equivalence of languages)

Two query languages L_1 and L_2 have the same expressive power if they can express the same queries.

(If the languages operate over different data inputs and outputs, have to normalize them before.)

Expressive Power of SPARQL

Three languages we will consider:

SPARQL

W3C Syntax and Semantics
(as in W3C Recommendation 15 Jan 2008).

Expressive Power of SPARQL

Three languages we will consider:

SPARQL

W3C Syntax and Semantics
(as in W3C Recommendation 15 Jan 2008).

SPARQL-S

W3C Syntax and Semantics. Only safe filters allowed.

Expressive Power of SPARQL

Three languages we will consider:

SPARQL

W3C Syntax and Semantics
(as in W3C Recommendation 15 Jan 2008).

SPARQL-S

W3C Syntax and Semantics. Only safe filters allowed.

SPARQL-C

SPARQL with compositional semantics
(as presented in this talk).

Expressive Power of SPARQL: Safe Patterns

What is the meaning of (P FILTER R) when $\text{var}(R) \not\subseteq \text{var}(P)$ (non-safe filters)?

Expressive Power of SPARQL: Safe Patterns

What is the meaning of (P FILTER R) when $\text{var}(R) \not\subseteq \text{var}(P)$ (non-safe filters)?

Example

Possible meanings of (?X name ?Y) FILTER (?Z $>$ 3)

Expressive Power of SPARQL: Safe Patterns

What is the meaning of (P FILTER R) when $\text{var}(R) \not\subseteq \text{var}(P)$ (non-safe filters)?

Example

Possible meanings of (?X name ?Y) FILTER (?Z $>$ 3)

1. Non-defined variable ?Z. (Error, False, empty set)

Expressive Power of SPARQL: Safe Patterns

What is the meaning of (P FILTER R) when $\text{var}(R) \not\subseteq \text{var}(P)$ (non-safe filters)?

Example

Possible meanings of (?X name ?Y) FILTER (?Z $>$ 3)

1. Non-defined variable ?Z. (Error, False, empty set)
2. All values of ?X, ?Y, ?Z such that the expression matches.

Expressive Power of SPARQL: Safe Patterns

What is the meaning of (P FILTER R) when $\text{var}(R) \not\subseteq \text{var}(P)$ (non-safe filters)?

Example

Possible meanings of (?X name ?Y) FILTER (?Z $>$ 3)

1. Non-defined variable ?Z. (Error, False, empty set)
2. All values of ?X, ?Y, ?Z such that the expression matches.
3. W3C uses the following:
   - IF the expression is inside an optional, e.g. P OPT ((?X name ?Y) FILTER (?Z >3)) and variable ?Z occurs in P, THEN (2.)
   - ELSE (1.)

Expressive Power of SPARQL: Safe Patterns

- Patterns with non-safe filter are rare cases.

Expressive Power of SPARQL: Safe Patterns

- Patterns with non-safe filter are rare cases.
- Patterns with non-safe filters are simulable with safe ones.

Expressive Power of SPARQL: Safe Patterns

- Patterns with non-safe filter are rare cases.
- Patterns with non-safe filters are simulable with safe ones.

Why not avoid them?

Expressive Power of SPARQL: Safe Patterns

- Patterns with non-safe filter are rare cases.
- Patterns with non-safe filters are simulable with safe ones.

Why not avoid them?

Theorem

SPARQL and SPARQL-S have the same expressive power.

Expressive Power of SPARQL: Safe Patterns

- Patterns with non-safe filter are rare cases.
- Patterns with non-safe filters are simulable with safe ones.

Why not avoid them?

Theorem

SPARQL and SPARQL-S have the same expressive power.

Proof idea

- *There exists generic procedure to translate non-safe queries into equivalent safe queries.*
- *It uses case-by-case W3C evaluation rules for non-safe queries.*

Expressive Power of SPARQL: Compositional semantics

- Compositional and denotational semantics are desirable.

Expressive Power of SPARQL: Compositional semantics

- Compositional and denotational semantics are desirable.
- W3C semantics of SPARQL has a complex three-level operational procedure for evaluating patterns.

Expressive Power of SPARQL: Compositional semantics

- Compositional and denotational semantics are desirable.
- W3C semantics of SPARQL has a complex three-level operational procedure for evaluating patterns.

Occam's razor: Why not keep things simple and clean?

Expressive Power of SPARQL: Compositional semantics

- Compositional and denotational semantics are desirable.
- W3C semantics of SPARQL has a complex three-level operational procedure for evaluating patterns.

Occam's razor: Why not keep things simple and clean?

Theorem

SPARQL-S and SPARQL-C have the same expressive power.

Expressive Power of SPARQL: Compositional semantics

- Compositional and denotational semantics are desirable.
- W3C semantics of SPARQL has a complex three-level operational procedure for evaluating patterns.

Occam's razor: Why not keep things simple and clean?

Theorem

SPARQL-S and SPARQL-C have the same expressive power.

Proof idea

The only non-trivial case is the semantics of patterns of the form $(P_1 \text{ OPT } (P_2 \text{ FILTER } C)$. *Just check both definitions coincide.*

Expressive Power of SPARQL: Relational Algebra

Interesting but not surprising:

Expressive Power of SPARQL: Relational Algebra

Interesting but not surprising:

Theorem

SPARQL-C and Relational Algebra have the same expressive power.

Expressive Power of SPARQL: Relational Algebra

Interesting but not surprising:

Theorem

SPARQL-C and Relational Algebra have the same expressive power.

Proof idea.

1. Use known equivalence between Relational Algebra and version of Datalog.
2. From SPARQL-C to Relational Algebra: idea of transformation was known, e.g., Cyganiak (to Relational Algebra), Polleres (to Datalog). Had to extend to bag semantics.
2. From Datalog to SPARQL-C key issue is sound translation of negation. □

Expressive Power of SPARQL: Relational Algebra

Interesting and surprising:

Expressive Power of SPARQL: Relational Algebra

Interesting and surprising:

Theorem

W3C SPARQL and Relational Algebra have the same expressive power.

Proof Idea.

Use previous results. □

Expressive Power of SPARQL: Relational Algebra

Interesting and surprising:

Theorem

W3C SPARQL and Relational Algebra have the same expressive power.

Proof Idea.

Use previous results. □

- Results hold for bag and set semantics.

Expressive Power of SPARQL: Some consequences

A. Domestic:

Expressive Power of SPARQL: Some consequences

A. Domestic:

- Expressive power of SPARQL (limitations and potentialities) completely clarified.

Expressive Power of SPARQL: Some consequences

A. Domestic:

- Expressive power of SPARQL (limitations and potentialities) completely clarified.
- Negation (difference) expressible in SPARQL.

Expressive Power of SPARQL: Some consequences

A. Domestic:

- Expressive power of SPARQL (limitations and potentialities) completely clarified.
- Negation (difference) expressible in SPARQL.
- Extension with ASK queries does not add expressive power.

Expressive Power of SPARQL: Some consequences

A. Domestic:

- Expressive power of SPARQL (limitations and potentialities) completely clarified.
- Negation (difference) expressible in SPARQL.
- Extension with ASK queries does not add expressive power.
- Could bring to SPARQL most of the machinery of basic SQL.

Expressive Power of SPARQL: Some consequences

A. Domestic:

- Expressive power of SPARQL (limitations and potentialities) completely clarified.
- Negation (difference) expressible in SPARQL.
- Extension with ASK queries does not add expressive power.
- Could bring to SPARQL most of the machinery of basic SQL.

B. Foundational:

Expressive Power of SPARQL: Some consequences

A. Domestic:

- Expressive power of SPARQL (limitations and potentialities) completely clarified.
- Negation (difference) expressible in SPARQL.
- Extension with ASK queries does not add expressive power.
- Could bring to SPARQL most of the machinery of basic SQL.

B. Foundational:

- SPARQL is a pattern matching version of SQL.

Expressive Power of SPARQL: Some consequences

A. Domestic:

- Expressive power of SPARQL (limitations and potentialities) completely clarified.
- Negation (difference) expressible in SPARQL.
- Extension with ASK queries does not add expressive power.
- Could bring to SPARQL most of the machinery of basic SQL.

B. Foundational:

- SPARQL is a pattern matching version of SQL.
- Only local queries expressible in SPARQL.

Expressive Power of SPARQL: Some consequences

A. Domestic:

- Expressive power of SPARQL (limitations and potentialities) completely clarified.
- Negation (difference) expressible in SPARQL.
- Extension with ASK queries does not add expressive power.
- Could bring to SPARQL most of the machinery of basic SQL.

B. Foundational:

- SPARQL is a pattern matching version of SQL.
- Only local queries expressible in SPARQL.
- Still waiting for the third query paradigm: SQL/Tables, XQUERY/Trees, ?/Graphs

Conclusions, Final Thougths

- RDF is becoming a very relevant data model. Develop it!
- We have a "convenience marriage" with SPARQL. Learn to love it. RDF and SPARQL are our assets. Be patient!
- Simplify, simplify, simplify. Keep everything (but your hope) minimal!
- Do not stop the search for "el Dorado". Missing graph features will be needed!
- Do not reinvent the wheel. Before designing new features or extensions for SPARQL, check if it was tried for SQL.
- SPARQL standardization + popularization of RDF + pervasiveness of social networks = explosive combination.

Conclusions, Final Thougths

- RDF is becoming a very relevant data model. Develop it!
- We have a "convenience marriage" with SPARQL. Learn to love it. RDF and SPARQL are our assets. Be patient!
- Simplify, simplify, simplify. Keep everything (but your hope) minimal!
- Do not stop the search for "el Dorado". Missing graph features will be needed!
- Do not reinvent the wheel. Before designing new features or extensions for SPARQL, check if it was tried for SQL.
- SPARQL standardization + popularization of RDF + pervasiveness of social networks = explosive combination.

Conclusions, Final Thougths

- RDF is becoming a very relevant data model. Develop it!
- We have a "convenience marriage" with SPARQL. Learn to love it. RDF and SPARQL are our assets. Be patient!
- Simplify, simplify, simplify. Keep everything (but your hope) minimal!
- Do not stop the search for "el Dorado". Missing graph features will be needed!
- Do not reinvent the wheel. Before designing new features or extensions for SPARQL, check if it was tried for SQL.
- SPARQL standardization + popularization of RDF + pervasiveness of social networks = explosive combination.

Conclusions, Final Thougths

- RDF is becoming a very relevant data model. Develop it!
- We have a "convenience marriage" with SPARQL. Learn to love it. RDF and SPARQL are our assets. Be patient!
- Simplify, simplify, simplify. Keep everything (but your hope) minimal!
- Do not stop the search for "el Dorado". Missing graph features will be needed!
- Do not reinvent the wheel. Before designing new features or extensions for SPARQL, check if it was tried for SQL.
- SPARQL standardization + popularization of RDF + pervasiveness of social networks = explosive combination.

Conclusions, Final Thougths

- RDF is becoming a very relevant data model. Develop it!
- We have a "convenience marriage" with SPARQL. Learn to love it. RDF and SPARQL are our assets. Be patient!
- Simplify, simplify, simplify. Keep everything (but your hope) minimal!
- Do not stop the search for "el Dorado". Missing graph features will be needed!
- Do not reinvent the wheel. Before designing new features or extensions for SPARQL, check if it was tried for SQL.
- SPARQL standardization + popularization of RDF + pervasiveness of social networks = explosive combination.

Conclusions, Final Thougths

- RDF is becoming a very relevant data model. Develop it!
- We have a "convenience marriage" with SPARQL. Learn to love it. RDF and SPARQL are our assets. Be patient!
- Simplify, simplify, simplify. Keep everything (but your hope) minimal!
- Do not stop the search for "el Dorado". Missing graph features will be needed!
- Do not reinvent the wheel. Before designing new features or extensions for SPARQL, check if it was tried for SQL.
- SPARQL standardization + popularization of RDF + pervasiveness of social networks = explosive combination.

Conclusions, Final Thougths

- RDF is becoming a very relevant data model. Develop it!
- We have a "convenience marriage" with SPARQL. Learn to love it. RDF and SPARQL are our assets. Be patient!
- Simplify, simplify, simplify. Keep everything (but your hope) minimal!
- Do not stop the search for "el Dorado". Missing graph features will be needed!
- Do not reinvent the wheel. Before designing new features or extensions for SPARQL, check if it was tried for SQL.
- SPARQL standardization + popularization of RDF + pervasiveness of social networks = explosive combination.

Comments, Questions, etc.

Thanks for your attention!

cgutierr@dcc.uchile.cl